Bachelor's thesis

in the Media Informatics degree program

Design and Evaluation of Audio-Analytical Methods for Real-Time Visualization of Musical Parameters in an Interactive Music Learning Application

submitted by

Erik Lindinger

Matrikelnummer: 44686

at the Hochschule der Medien Stuttgart on the

26th August 2026

for the attainment of the academic degree of Bachelor of Science

First Examiner: Prof. Dr. Stefan Radicke

Second Examiner: Willi Schorrig

Ehrenwörtliche Erklärung

Hiermit versichere ich, Erik Lindinger, ehrenwörtlich, dass ich die vorliegende Bachelorarbeit mit dem Titel: „Design and Evaluation of Audio-Analytical Methods for Real-Time Visualization of Musical Parameters in an Interactive Music Learning Application“ selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ebenso sind alle Stellen, die mit Hilfe eines KI-basierten Schreibwerkzeugs erstellt oder überarbeitet wurden, kenntlich gemacht. Die Arbeit ist noch nicht veröffentlicht oder in anderer Form als Prüfungsleistung vorgelegt worden.
Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO, § 23 Abs. 2 Master-SPO (Vollzeit)) einer unrichtigen oder unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen.

Ort, Datum	Unterschrift

Kurzfassung

Interaktive Musiklern-Anwendungen setzen zunehmend auf visuelles Echtzeit-Feedback, um unsichtbare Audioeigenschaften wie Tonhöhe, Tondauer und Klangfarbe für Lernende wahrnehmbar zu machen. Die dafür verwendeten Audioanalyseverfahren werden jedoch selten unter den praktischen Bedingungen einer realen Anwendung evaluiert. Diese Arbeit untersucht, welche Audioanalyseverfahren sich am besten für die Echtzeit-Visualisierung von Tonhöhe und Tondauer in einer interaktiven Musiklern-Anwendung eignen, unter Berücksichtigung von Signalgenauigkeit, Echtzeitfähigkeit und Verständlichkeit für die User. Fünf Verfahren, RMS/Envelope Following, FFT-basiertes Dominant Frequency Picking, Chroma Feature Extraction, STFT Band Energy und Onset Detection, wurden in Tone, einer auf Unity basierenden Musiklern-Anwendung, implementiert und sowohl technisch, anhand kontrollierter Aufnahmen eines einzelnen sowie mehrerer Instrumente, als auch empirisch, durch eine Within-Subjects-Nutzerstudie ($n = 15$) im Vergleich zum bestehenden statischen Visualisierungssystem von Tone, evaluiert.
Die technische Evaluation zeigt, dass kein einzelnes Verfahren für sich genommen präzise genug ist, um tonhöhenbasierte Visualisierungen zuverlässig zu steuern, sobald sich die Klangfarbe des Instruments ändert. Chroma Feature Extraction kommt dem am nächsten, verallgemeinert jedoch nicht zuverlässig über Instrumente hinweg, während Onset Detection als einziges Verfahren unabhängig von der Klangfarbe zuverlässig bleibt, allerdings ohne jegliche Tonhöheninformation zu liefern. Keines der fünf Verfahren ist jedoch durch die Echtzeitfähigkeit limitiert. Alle bewegen sich komfortabel innerhalb des gemessenen Zeitbudgets der Audioverarbeitung. Auf Basis dieser Erkenntnisse wurde ein kombinierter Visualisierungsansatz implementiert, der bei der Farbgebung auf die tatsächliche Tonhöhe statt auf einen einzelnen erkannten Wert zurückgreift, und mit Nutzenden evaluiert. Diese empfanden es als hilfreich zur Identifikation von Tönen und nicht als störend, wobei seine Notwendigkeit jedoch weiterhin offen blieb. Die Ergebnisse legen nahe, dass eine audioanalysebasierte Echtzeit-Visualisierung musikalischer Parameter technisch bereits heute umsetzbar ist, sofern das System nicht von der Genauigkeit eines einzelnen Erkennungsverfahrens abhängig gemacht wird.

Stichwörter: Echtzeit-Audioanalyse, Tonhöhenerkennung, Musiklernen, Visuelles Feedback, Chroma Features, Onset Detection, Klangfarbe, Unity

Abstract

Interactive music-learning applications increasingly rely on real-time visual feedback to make invisible audio properties ,as pitch, duration and timbre, perceptible to learners, but the audio-analysis methods commonly used to drive such feedback are rarely evaluated against the practical constraints of an actual application. This thesis investigates which audio-analysis methods are best suited for real-time visualization of pitch and note duration in an interactive music-learning application, considering signal accuracy, real-time performance and interpretability for the user. Five methods, RMS/envelope following, FFT dominant-frequency picking, chroma feature extraction, STFT band energy and onset detection, were implemented within Tone, a Unity-based music-learning application and evaluated both technically, using controlled single-instrument and cross-instrument audio captures and empirically, through a within-subjects user study ($n = 15$) comparing the resulting visualization against Tone's existing static visual system.

The technical evaluation shows that no single method is accurate enough on its own to drive pitch-based visuals once instrument timbre varies. Chroma feature extraction comes closest but does not generalize across instruments, while onset detection is the only method that remains reliable regardless of timbre, at the cost of carrying no pitch information. None of the five methods, however, is limited by real-time performance. All operate comfortably within the measured audio-processing time budget. Building on these findings, a combined visualization approach that falls back on ground-truth pitch information rather than any single detected value was implemented and evaluated with users, who found it helpful for identifying notes and not distracting, though its necessity remained genuinely in question. These results suggest that real-time, audio-analysis-driven visualization of musical parameters is technically feasible today, provided the system does not depend on any single detection method in isolation.

Keywords: real-time audio analysis, pitch detection, music learning, visual feedback, chroma features, onset detection, timbre, Unity

Contents

List of Abbreviations viii

List of Figures ix

List of Tables x

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Problem Formulation . . . 1
- 1.3 Thesis Objective . . . 1
- 1.4 Research Question . . . 2
- 1.5 Methodology . . . 2

2 Fundamentals and Theory behind the Audio Analytic Methods 4
- 2.1 Tonal Structure . . . 4
  - 2.1.1 Pitch, Note and Frequency . . . 4
  - 2.1.2 Harmonic Series and Overtones . . . 5
  - 2.1.3 Timbre . . . 5
- 2.2 Digital Audio Signals . . . 5
  - 2.2.1 Sampling and Pulse-Code Modulation . . . 6
  - 2.2.2 Fast Fourier Transform and Frequency Analysis . . . 6
  - 2.2.3 Short-Time Fourier Transform and Spectrograms . . . 7
- 2.3 Audio Analysis Methods . . . 8
  - 2.3.1 RMS and Envelope Following . . . 8
  - 2.3.2 Chroma Features . . . 10
  - 2.3.3 Onset Detection . . . 11
- 2.4 Real-Time Constraints in Unity . . . 11
  - 2.4.1 Audio Thread vs Main Thread in Unity . . . 12
  - 2.4.2 Audio-Visual Synchronisation . . . 13
  - 2.4.3 Computational Budget . . . 13
- 2.5 Visual Feedback in Music Learning . . . 13

3 Concept for the Audio Visualization System 15
- 3.1 Requirements . . . 15
  - 3.1.1 Functional requirements . . . 15
  - 3.1.2 Non-functional requirements . . . 15
- 3.2 Selection of Analysis Methods . . . 15
- 3.3 System Architecture . . . 18
  - 3.3.1 Audio Capture . . . 18
  - 3.3.2 Feature Extraction . . . 19
  - 3.3.3 EvaluationLogging . . . 19

3.4 Mapping Musical Parameter . . . 20
3.4.1 Audio Features . . . 20
3.5 Visualization Concepts . . . 21
3.5.1 Particle System . . . 22

4 Implementation **23**
4.1 Unity Architecture . . . 23
4.1.1 Component Structure . . . 23
4.1.2 Shared Data Contracts . . . 24
4.1.3 Extractor and Responder Registration . . . 24
4.1.4 Note Lifecycle Detection . . . 24
4.1.5 Shared Per-Clip Analysis Cache . . . 24
4.2 Audio Analysis . . . 25
4.2.1 RMS/Envelope . . . 25
4.2.2 FFT . . . 25
4.2.3 Chroma . . . 25
4.2.4 Onset . . . 25
4.2.5 STFT . . . 26
4.3 Mapping System . . . 26
4.3.1 Responder Interface and Selection . . . 26
4.3.2 Motivation . . . 26
4.3.3 Design . . . 27
4.3.4 Scope Note . . . 28
4.4 Real-Time Visualization . . . 28
4.4.1 Audio-Thread to Main-Thread Handoff . . . 28
4.4.2 Per-Frame Update Cycle . . . 28
4.4.3 Avoiding Visual Jitter . . . 28
4.5 Performance Optimization . . . 29
4.5.1 Deliberate Optimizations Already in Place . . . 29
4.5.2 Remaining Costs Worth Naming Explicitly . . . 29
4.5.3 Sampled vs. Full-Resolution Logging . . . 29

5 Evaluation **30**
5.1 Experimental Setup . . . 30
5.2 Implementation of the Evaluation . . . 33
5.3 Technical Evaluation . . . 33
5.3.1 Pitch-Tracking Accuracy (Single Instrument) . . . 33
5.3.2 Chroma Correction Generalization Across Instruments . . . 38
5.3.3 Timbre Robustness Across Instruments and the Combined Responder's Fix 39
5.3.4 Real-Time Performance . . . 41

5.4 User Survey 42
5.4.1 Paired Preference Results 43
5.4.2 Color-Specific Ratings 44
5.4.3 Breakdown by Prior Music Experience 45
5.5 Evaluation and Discussion 46

6 Conclusion and Future Work **48**
6.1 Conclusion 48
6.2 Future Work 48

7 Use of AI Tools **49**

References **51**

Acknowledgements **54**

List of Abbreviations

ADSR	Attack, Decay, Sustain, Release
API	Application Programming Interface
CSV	Comma-Separated Values
CQT	Constant-Q Transform
CV	Coefficient of Variation
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing
DTFT	Discrete-Time Fourier Transform
FFT	Fast Fourier Transform
MIDI	Musical Instrument Digital Interface
PCM	Pulse-Code Modulation
RMS	Root Mean Square
SD	Standard Deviation
STFT	Short-Time Fourier Transform

List of Figures

2.1 Chromatic Circle . . . 5
2.2 Spectrogram with different window sizes . . . 7
2.3 Computation of an energy-based novelty function . . . 9
2.4 Waveform and energy-based novelty function . . . 9
2.5 ADSR envelope and spectrogram of piano and violin . . . 10
2.6 Double-buffering handoff between audio and main thread . . . 12
3.1 Timbre robustness across instruments . . . 17
3.2 Pipeline of the system architecture . . . 18
4.1 UML diagram of the system architecture. . . . 23
5.1 Side-by-Side Comparison . . . 31
5.2 Old System Note Sequence . . . 31
5.3 New System Note Sequence . . . 32
5.4 RMS/envelope peak value by note, marimba only. No pitch-ordered separation is visible. . . . 34
5.5 FFT dominant-frequency detection by note, marimba only. . . . 35
5.6 Mean normalised chroma feature vectors . . . 36
5.7 STFT magnitude spectrogram . . . 37
5.8 Spectral centroid (magnitude-weighted mean frequency) at five fixed post-onset frames, for four notes played on the same instrument. . . . 38
5.9 Guitar vs. Marimba Comparison . . . 39
5.10 Onset latency . . . 41
5.11 Double-buffer lock duration . . . 42
5.12 Preference Results . . . 44
5.13 Color-Specific Ratings . . . 45
5.14 Breakdown by Prior Music Experience . . . 46

List of Tables

5.1 FFT-dominant frequency results 35
5.2 Chroma pitch class detection, marimba only 36
5.3 Chroma pitch class detection by instrument (fixed note D) 38
5.4 Onset detection reliability and latency by instrument (fixed note D) 40
5.5 RMS / Envelope – raw vs. normalized drive 40
5.6 FFT dominant frequency by instrument (fixed note D) 41
5.7 STFT mid/high band energy by instrument (fixed note D) 42
5.8 Double-buffer lock duration 43
7.1 AI-based tools used in the course of this thesis. 50

1 Introduction

1.1 Motivation

Learning to recognize pitch and rhythm is one of the central challenges for music beginners and visual feedback has long been proposed as a way to make these abstract auditory concepts more tangible. Real-time visual feedback has been shown to improve pitch-matching accuracy and retention compared to auditory-only or no feedback [2, p. 10], [2, pp. 12–13]. However, it remains unclear how much the accuracy of this visual feedback itself shapes the users experience and how different real-time audio-analysis procedures compare in producing it. This thesis investigates that question by implementing and comparing several audio-analytic methods within an interactive music learning application, evaluating both the accuracy of the analysis and how user's subjectively perceive the resulting visualization.

1.2 Problem Formulation

The current music pedagogy for learning an instrument is often criticized for overemphasizing abstract theory and notation at the expense of practical, ear-based musicianship. The usual way of learning pitch, rhythm and beat is by theory and practice with an instrument [7, pp. 109–111]. Visualization as a support is not often used while Game Engines and realtime-audio analytics add new opportunities such as how to display musical parameters dynamicly and interactively [2, pp. 7]. Many music visualizations are just based on simple frequency representation or static visual methods [10, p. 143:24–25]. The comparison of different audio analysis methods regarding their suitability for real-time musical visualization has so far been investigated only to a limited extent, particularly in the context of interactive learning systems [10, p. 143:24]. The music learning app Tone, developed by Willi Schorrig, is in his final steps for launch. It is based on a Musical Instrument Digital Interface (MIDI)-like approach in which musical phrases are generated using prepared instrument samples and Attack, Decay, Sustain, Release (ADSR) envelopes. Users correctly arrange notes on a scale or timeline based on their pitch and rhythmic spacing [17, pp. 1–2]. As part of this project, the existing application is to be extended with a dynamic visualization system. This thesis does not evaluate long-term learning outcomes, as measuring such effects would require a longitudinal study beyond its scope. Instead, it focuses on the technical accuracy of the visualization and users immediate subjective perception of it. Both are necessary preconditions before the learning effects could be meaningfully studied.

1.3 Thesis Objective

Aim of this work is to build a concept, implementation and evaluation of a modular realtime visualization system for audio parameters in the framework of an interactive music learning app in Unity.

1.4 Research Question

Which audio analysis methods are best suited for the real-time visualization of pitch and note duration in an interactive music-learning applications, considering signal accuracy, real-time performance and interpretability for the user?

1.5 Methodology

As part of this project, the existing music learning app is being enhanced with a modular visualization system. To this end, several audio analysis techniques are implemented and compared. The focus is on a selected group of suitable methods:

- FFT (Fast Fourier Transform)
- STFT (Short-Time Fourier Transform)
- Chroma Features
- Onset Detection
- RMS (Root Mean Square)

The selected methods are used to extract and visually represent musical characteristics, such as frequency, rhythm, dynamics and pitch classes in real time. Building on this, various visualization concepts are implemented, including:

- Particle systems
- Color mapping
- Note duration representation

Additionally, direct musical data from the existing MIDI-like system is visualized and compared with the results obtained via the signal analysis techniques. This comparison forms the basis for evaluating signal accuracy. A user study is conducted for evaluation purposes. The five analysis methods are intentionally decoupled from full gameplay for controlled, one-variable-at-a-time comparison, where each analysis method gets its own isolated responder. The user gets to experience for the evaluation, the sixth responder, which is a single, combined responder (Section 4.3) that fuses multiple analysis signals into one practical, timbre-robust representation against the existing static visual system. Participants rate both conditions on understandability and perceived synchronicity, which operationalize the research questions interpretability and performance dimensions, respectively. Technical metrics such as latency, stability and real-time capability provide a complementary, objective measure of performance.
To capture broader qualitative impressions beyond the core research dimensions, participants additionally rate Musical Coherence, Visual Appeal and Overall User Experience.
Having outlined the approach, the remainder of this thesis is structured as follows: Chapter 2 covers the theoretical fundamentals of audio analysis and music visualization; Chapter 3 presents

the concept; Chapter 4 describes the implementation; Chapter 5 presents the evaluation and results; Chapter 6 discusses the findings and concludes.

2 Fundamentals and Theory behind the Audio Analytic Methods

The visualization of the audio happens by mapping musical parameters which the audio analytic methods prepare. To understand how the audio analytic processes and mapping is working, the basics of music, how it can be separated and how each component defines the sound and can be used as driving parameters, needs to be understood.

2.1 Tonal Structure

Describing music and separating sound and tone in different structural parts is difficult because it is important to know in which context the music needs to be interpreted. For the music learning app it is significant to define music from the user perspective, but for analysing and comparing on a digital level a scientific approach is required.

2.1.1 Pitch, Note and Frequency

A note is a musical symbol that represents a frequency of a tone which can be categorized into different letters that humans can convert into written language on sheet music to read and play a tone. A pitch class is the equivalent to a note, so the pitch classes can be classified in twelve equal tempered scaled notes. They are represented by the seven letters C, D, E, F, G, A and B. The remaining five pitch classes, C#, D#, F#, G# and A#, are notes that are raised by a semitone and are equivalent to the black keys on a piano or keyboard. A semitone is the smallest interval. The ratio between adjacent semitones is constant ($2^{1/12}$), but because pitch grows exponentially, the absolute frequency gap between adjacent semitones is not constant, so it widens at higher pitches. The pitch represents the sound at how fast it vibrates and is measured in Hertz [13, pp. 3–5]. The note A4 has a fundamental frequency of 440Hz and serves as a reference. If two frequencies of the same pitch class are perceived as similar, they differ by a power of two, which is why the term octave was established. For example, the pitches A3 (220Hz), A4 (440 Hz) and A5 (880 Hz) sound similar. Furthermore, the perceived distance between A3 and A4 is the same as between A4 and A5, which shows that human pitch perception is logarithmic in nature [13, pp. 21–22], as illustrated by the pitch helix in Figure 2.1.

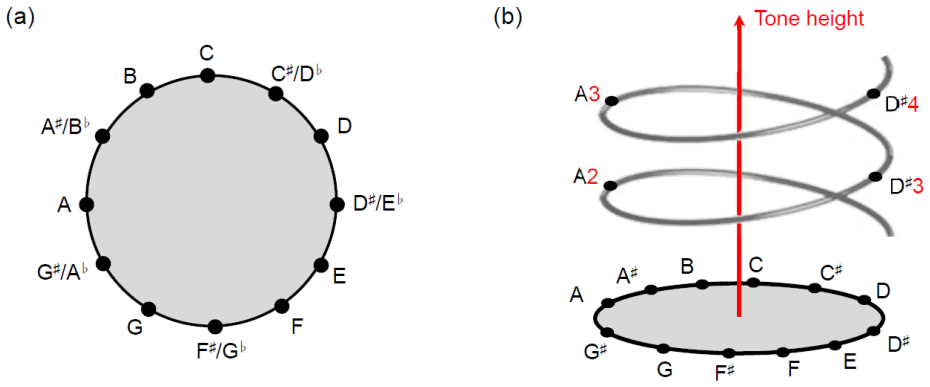


Figure 2.1: (a) The chromatic circle of the twelve pitch classes. (b) The pitch helix, which separates pitch class (position around the circle) from tone height (position along the vertical axis), illustrating why pitches an octave apart, such as A2/A3 or D#3/D#4, are perceived as similar. Source: [13, p. 5].

The tone can also be separated in different harmonic series and overtones.

2.1.2 Harmonic Series and Overtones

The harmonic series can not be separated from the fundamental and is a distinct structure from the twelve pitch classes shown in Figure 2.1. It is an arithmetic series which is always constant and equal like ω, 2ω, 3ω and so on. Most instruments were designed to have partials that are close to being harmonics and try to have almost no inharmonicity. Overtones are all frequencies that are higher than the fundamental frequency. To summarize, all harmonics are overtones, except for the first harmonic, which is the fundamental, but not all overtones are harmonics. Some pitched instruments, like marimba or bells contain nonharmonic partials, but they still give the ear a good sense for the pitch [13, pp. 23–30].

2.1.3 Timbre

Timbre is the perceptual attribute that lets two notes at the same pitch and loudness still sound different, e.g. a piano vs violin playing an A4. It depends on which harmonics are present and how loud each is relative to the others, as well as how that harmonic content evolves over time, such as the shape of the attack and decay. Also the physical traits of an instrument determine which harmonics are present and how loud each one is relative to the others.
On top of those magnitude spectrums there can be drawn a shape which connects the harmonic peaks called spectral envelopes. These together determine the timbre of a sound [12].

2.2 Digital Audio Signals

The digital audio signal is a representation of an analog audio signal in a digital form. Since the digital audio signal is a discrete representation of the continuous analog audio signal, it is necessary to understand the process of sampling and quantization, which are fundamental

concepts in digital audio processing. Also a general understanding of the FFT and STFT is required to understand how the audio signal can be analyzed and processed in the digital domain.

2.2.1 Sampling and Pulse-Code Modulation

The sampling process of audio signals is an important step in having a useful form of data which can be analyzed further. The waveforms of analog audio signals are sampled over time. A higher sample rate increases the Nyquist frequency and reduces aliasing risk. The sample rate will then be measured in Hertz [13, p. 52]. The Nyquist Frequency is the highest frequency which can be recorded at any sample rate and is half of the sample rate [13, p. 51]. The most common standard is a sampling rate of 44,1kHz which is used for CD's and music. So for 44.1kHz sample rate in our case, the capture limit is 22.05kHz. A frequency above the Nyquist Frequency cannot be captured correctly at all. It folds back onto a lower alias frequency that is, once sampled, indistinguishable from a genuine signal actually at that frequency, so the original frequency cannot be recovered even in principle. This aliasing produces audible artefacts, often perceived as inharmonic to the fundamental frequency [13, pp. 60–61]. The OnAudioFilterRead-Unity API gives access to the Pulse-Code Modulation (PCM) data. PCM is the encoding of digital audio. It is a representation of analog audio waveform as a sequence of discrete numeric values sampled at regular intervals. Each sample gives us the information of the amplitude of the sound wave at a given time. Unity then normalizes those values into floats into a range between [-1 and 1] [21].

2.2.2 Fast Fourier Transform and Frequency Analysis

The Fast Fourier Transform (FFT) is an algorithm often used in signal processing to efficiently compute the Discrete Fourier Transform (DFT), defined for a sequence $x[n]$ of length N as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk}, \quad k = 0, 1, \ldots, N-1 \tag{2.1}$$

where $W_N = e^{-j2\pi/N}$ [15, p. 751]. Rather than the $O(N^2)$ operations a direct DFT would need, the FFT exploits the periodicity and symmetry of the twiddle factors W_N^{nk} to recursively split the computation, reducing it to approximately $N \log_2 N$ operations [15, pp. 746, 751].
A Hann Window is used because the FFT implicitly assumes the analyzed segment is periodic. If the signal does not taper smoothly to zero at the edges of the window, spectral leakage appears, where energy from one frequency smears into neighboring frequency bins and distorts the result [15, p. 828]. The Hann window prevents this by shaping the segment so it fades in and out to zero at the start and end, defined as $w_{\text{Hann}}[n] = 0.5 - 0.5\cos(2\pi n/M)$ for $0 \leq n \leq M$ [15, p. 851].

2.2.3 Short-Time Fourier Transform and Spectrograms

The Short-Time Fourier Transform (STFT) is an analytic method which is the basis of a wide range of speech analysis, coding and synthesis systems. It is computed as a sequence of DTFTs, each applied to a short time window of the signal as that window slides forward in time. It can also be interpreted as a filter bank of bandpass filters, each centered at a frequency of interest, with the output of each filter observed as it changes over time [16, pp. 43–44]. The STFT determines the position of the window with two variables, $\hat{n}$ as a discrete-time index for position and $w(\hat{n} - m)$ for the window. The signal outside that window gets zeroed out. The small windowed chunk gets transformed with the DTFT, which gives us one spectrum near the time $\hat{n}$. Now with each slide of $\hat{n}$ a new spectrum is obtained. With $\hat{n}$ and $\hat{\omega}$, the frequency within the window, the two variables the STFT needs to form a 2D map are complete [16, pp. 43–44]. This 2D map is displayed visually as a spectrogram, with time on one axis and frequency on the other [16, p. 46].

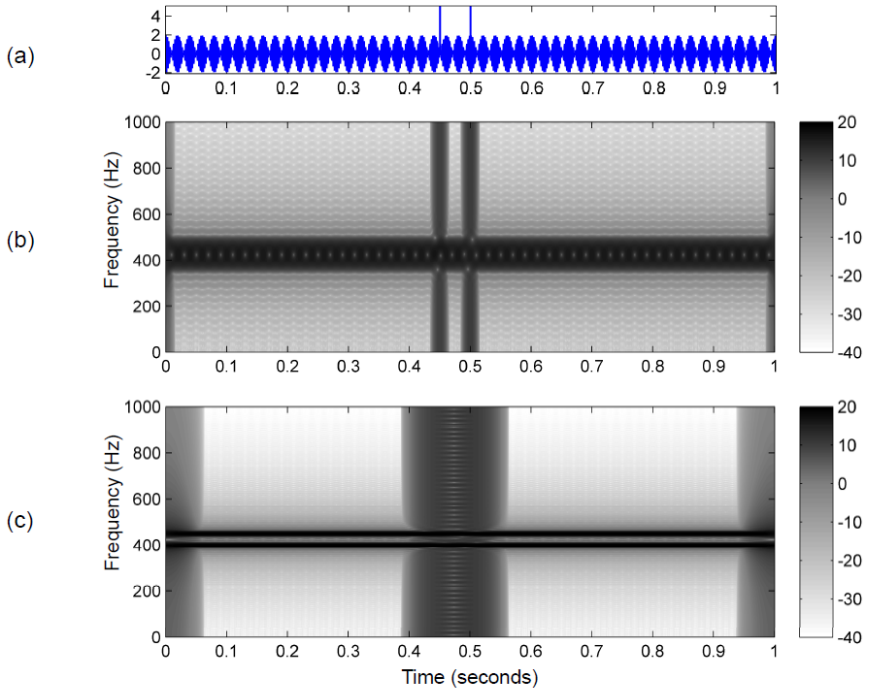


Figure 2.2: (a) Captured Signal. Spectrogram of the same signal computed with (b) a short Hann window (32 ms) and (c) a long Hann window (128 ms). Source: [13, p. 101].

The hop size, window function and centroid calculation just indicate how smooth or noisy the spectrogram looks. Hop size is described purely as a time-sampling-density parameter, independent of frequency resolution. The Hann function shapes how much spectral leakage is generated. The sharper the window shape, the cleaner the peaks, but the more they ring. The smoother the window, the less ringing, but the peaks get wider. The centroid calculation determines the centroid, which is an amplitude-weighted average across frequency bins [9, p. 2]. When the window is too short, harmonics can smear across bins (see Figure 2.2), which

manipulates the average and disturbs the timbre change. If the window is too long, the centroid smooths out and loses sharp, clean transitions at the start of an onset or at an other important moment [9, p. 4].

2.3 Audio Analysis Methods

The following methods are used in this thesis to extract musical parameters from the audio signal: RMS, Chroma Features and Onset Detection.

2.3.1 RMS and Envelope Following

A common method to detect the power of a sound and when it starts is the RMS. It takes the sliding window approach of the STFT but does not perform an FFT on each window frame chunk [13, pp. 27–30, 312–314]. The fundamental computation behind RMS and envelope following is the short-time energy of a windowed segment,

$$E_{\hat{n}} = \sum_{m=-\infty}^{\infty} \left(x[m]w[\hat{n} - m]\right)^2 = \phi_{\hat{n}}[0], \tag{2.2}$$

from which RMS itself follows as RMS $= \sqrt{E/N}$ [16, p. 42]. Because this only requires squaring and summing samples, it is $O(N)$ with no complex multiplications, unlike the FFT's $O(N \log N)$ with twiddle factors, making it considerably cheaper per operation. It describes how much energy is present at a certain point in time, also referred to as the average loudness over the buffer and is considered a more accurate way of measuring perceived loudness than instantaneous peak level [6, pp. 243–244]. Figure 2.3 shows this local energy computation applied to a waveform, together with the derivative and half-wave rectification steps used to build an onset novelty function, covered in the following subsection on Onset Detection.

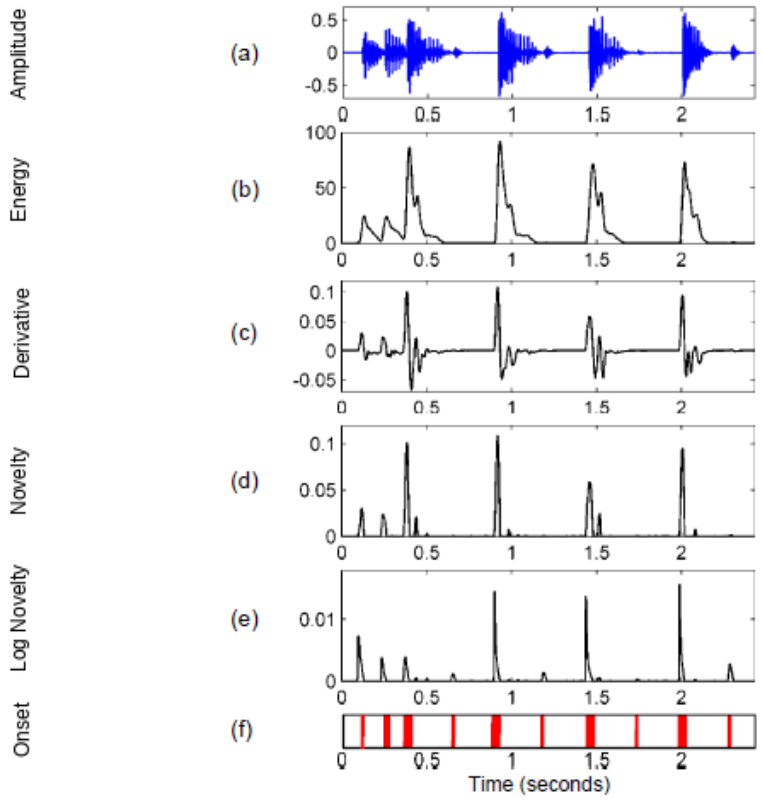


Figure 2.3: Computation of an energy-based novelty function: (a) waveform, (b) local energy function, (c) discrete derivative, (d) novelty function after half-wave rectification, (e) novelty function based on logarithmic energy, (f) annotated note onsets. Source: [13, p. 313].

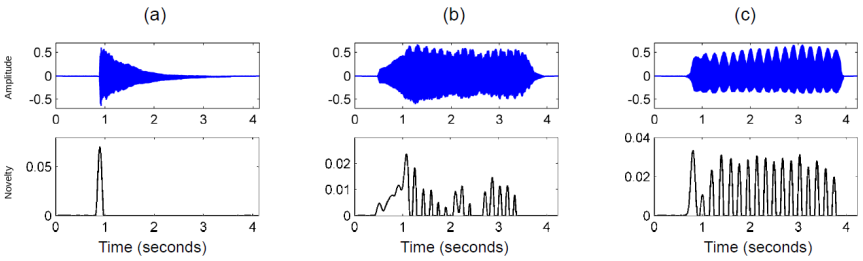


Figure 2.4: Waveform and energy-based novelty function of the note C4 (261.6 Hz) played by different instruments: (a) piano, (b) violin, (c) flute. Source: [13, p. 314].

Figure 2.4 shows this same energy-based approach applied to the same note (C4, 261.6 Hz) played by piano, violin and flute, illustrating how the resulting envelope shape already differs by instrument even before any dedicated timbre analysis is applied. The envelope extracted from a waveform is commonly described by four stages: Attack, Decay, Sustain and Release, together abbreviated as ADSR. Figure 2.5 shows this decomposition for a piano and a violin playing the same note. The piano's envelope rises quickly (attack), decays rapidly (decay) into a long,

quiet tail (sustain) and fades out (release), while the violin's envelope has a comparable attack but then holds a sustained, oscillating plateau for as long as the note is bowed, before its own release. The accompanying spectrograms also show that the piano's harmonics steadily decay in amplitude over time, whereas the violin's harmonics remain present and continuously fluctuate in strength, which is a further expression of the timbre difference discussed in Section 2.1.3.

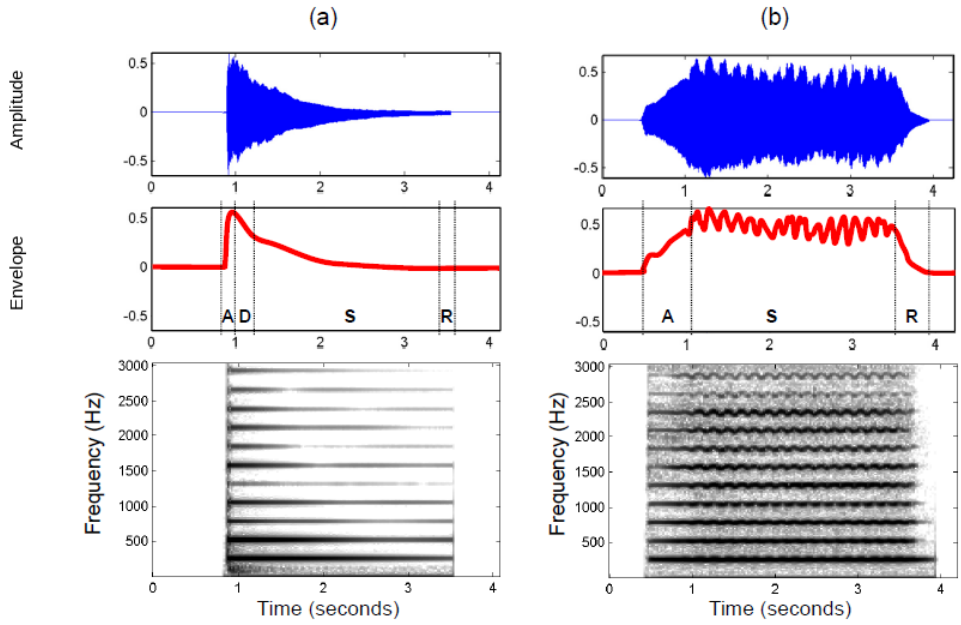


Figure 2.5: Waveform, ADSR-labeled amplitude envelope and spectrogram for the note C4 (261.6 Hz) played by (a) piano and (b) violin. Source: [13, p. 28].

2.3.2 Chroma Features

As discussed earlier in 2.1.1, a pitch class is the equivalent to a note, so the pitch classes can be classified into twelve equal-tempered scaled notes (see Figure 2.1). Through the logarithmic frequency transformation with the FFT as its base, each frequency bin of the magnitude spectrum is mapped to its pitch class via

$$\text{pitchClass} = \left(12 \cdot \log_2\left(\frac{f}{C_0}\right)\right) \bmod 12 \tag{2.3}$$

[13, p. 121]. The magnitudes of all bins sharing the same pitch class are then summed, folding the full magnitude spectrum into a single 12-element chroma vector, one pooled value per pitch class. C_0 (16.352 Hz) is used as a reference, which is derived from the anchor point A4 (440 Hz), MIDI note 69, via

$$F_{\text{pitch}}(p) = 2^{\frac{p-69}{12}} \cdot 440 \tag{2.4}$$

[13, p. 122, eq. 3.2]. The 2nd harmonic of G4 (784 Hz = G5) folds back into the same G chroma bin as the fundamental (392 Hz = G4). Overtones reinforce the correct pitch class

rather than confusing the detector. This is the key advantage over simple FFT peak-picking, which reports whichever frequency bin has the highest magnitude as the dominant frequency and that is often not the fundamental [13, p. 128].

2.3.3 Onset Detection

The Onset Detection is a different approach to analyzing a sound or a note. In comparison to the Chroma that analyzes the sound attributes of what it is made of and which category like pitch class it can be classified into, the Onset method tries to figure out when a new sound event starts [13, p. 311]. The Onset method always consists of three steps. It converts a raw signal into a feature which jumps when a sound begins [13, pp. 311–312], [1, p. 1037]. After that it takes the discrete derivative, also described as the difference between consecutive frames, of the STFT magnitude bins and filters out all the negative values, which is called half-wave rectification. Then those rectified per-bin values get summed across all frequency bins [13, p. 316, eq. 6.6], [1, p. 1038, eq. 5]. The resulting value is called spectral flux and is defined as:

$$\Delta_{\text{Spectral}}(n) := \sum_{k=0}^{K} \Big| Y(n+1,k) - Y(n,k) \Big|_{\geq 0} \tag{2.5}$$

which Bello et al. calls spectral difference, from STFT magnitude bins [1, p. 1038]. The spectral flux measures how fast the power spectrum of a signal changes over time [13, p. 315]. Then the peak-picking process takes place. In this step it scans for local maxima which are above a fixed or adaptive threshold [1, p. 1043]. Those are then the timings of new notes or sound events. The RMS is just energy detection and can miss smaller new notes that are played, so the spectral flux is more sensitive to notes that pure energy-based methods might miss [13, p. 315]. The new notes are showing up as a broadband jump across many bins even if the aggregate energy does not change much [1, p. 1038].

2.4 Real-Time Constraints in Unity

Since the implementation of the audio analytic system takes place within the Unity game engine on which the Tone app runs, it is important to understand the constraints of the game engine [17]. One priority for the audio visual system is real-time audio processing of notes, which has a heavy timing budget in comparison to the typical game loop. At 44.1 kHz with a 1024-sample buffer, the estimated time to finish the processing of it before the next buffer is needed is 23ms. The equation to calculate the time of a buffer is shown in 2.6 where N_{samples} is the number of samples in the buffer and f_s is the sample rate.

$$t_{\text{buffer}} = \frac{N_{\text{samples}}}{f_s} \tag{2.6}$$

If the processing of the buffer is not finished in that estimated time, audible glitches will occur [21]. That is why audio callbacks run on their own thread and do not share their computing with the main thread [21].

2.4.1 Audio Thread vs Main Thread in Unity

Figure 2.6 gives an overview of the double-buffering handoff between the audio thread and the main thread described in this section. For this the `OnAudioFilterRead` Unity-API, which runs on Unity's internal audio thread, comes into play. The Unity's documentation explains what is safely allowed inside and what is completely restricted. Most Engine API calls are not thread-safe for the audio thread with some exceptions like `Debug.Log`. It never got officially confirmed, but since the `Debug.Log` function still appears in a code example in the documentation of `OnAudioFilterRead`, it can be assumed that it is safe to use. The audio thread writes samples every ~20 ms when `OnAudioFilterRead` gets called into the `captureBuffer` [21].
The main thread works with the `readyBuffer`, a buffer which gets copies of the `captureBuffer`. Since the `readyBuffer` is shared by the main and audio thread, a lock is needed to prevent two threads from working on the same buffer simultaneously or it results in torn reads. While unlocked, the audio thread writes its samples into the `captureBuffer` and when completed, it does a quick copy into the `readyBuffer`. Then it gets locked and the main thread copies out of the `readyBuffer` and immediately processes the samples. The lock is held only for microseconds relative to the 23,220µs DSP buffer period, which is measured directly and reported in Section 5.3.4.

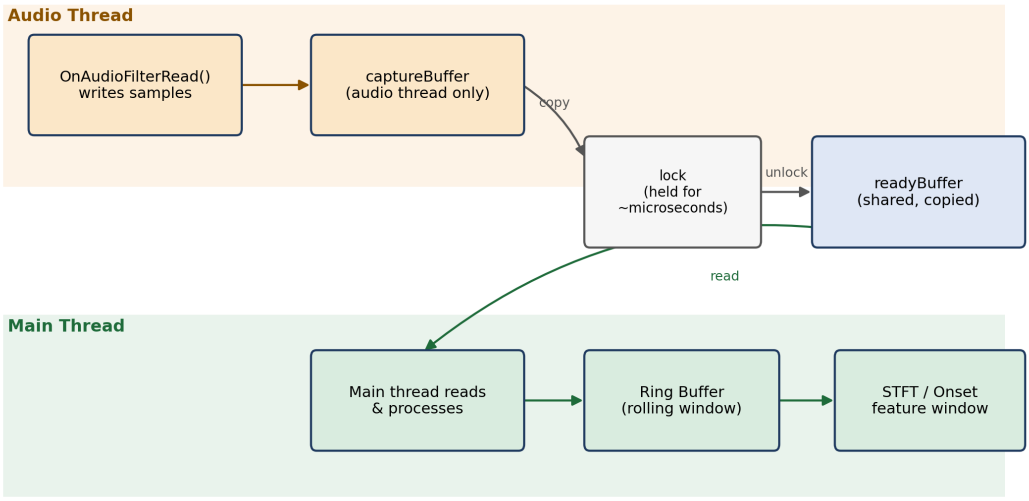


Figure 2.6: Double-buffering handoff between the audio thread and the main thread. The audio thread writes samples into `captureBuffer`, which is briefly locked and copied into the shared `readyBuffer`. The main thread reads from `readyBuffer` and feeds the samples into a ring buffer that provides the rolling window used by the STFT and Onset Detection methods.

This is called a Double Buffer. For the STFT method and the Onset Detection method, which is also based on the STFT, a rolling window is required. A `ringbuffer` handles this task. It runs completely on the main thread, but gets fed beforehand by the `OnAudioFilterRead` `readyBuffer`. The ring buffer introduces one additional frame of latency on top of the audio

thread's own latency.

2.4.2 Audio-Visual Synchronisation

The audio thread and the main thread run on separate clocks [21]. Audio advances in fixed buffer-size chunks, while rendering on the main thread runs once per frame at a variable rate. Because `Update()` runs once per rendered frame at a variable rate, the gap between when a given audio buffer was captured and when `Update()` actually processes it varies from frame to frame. `Time.time`, the main thread's own built-in clock, only updates once per frame and cannot measure that gap on its own. Without an explicit timestamp carried alongside the buffer, this offset can only be assumed rather than measured. And an assumed, fixed offset does not hold once the render rate drifts from its target. A common timestamp on both sides is therefore necessary to align a given audio event with the correct rendered frame rather than an inconsistent later one. Unity's audio-clock `dspTime` is suited to this because it is sample-accurate and tied directly to the audio system, whereas `Time.time` only updates once per rendered frame and is therefore already coarsened by the same variable frame rate it would need to compensate for [20]. `dspTime` is captured once per audio callback (`AudioSettings.dspTime`) when the shared `AudioBuffer` is built and every extractor copies this same value onto its output frame rather than computing its own, so the timestamp is consistent across all five methods rather than something individual extractors like STFT or `OnsetDetector` determine independently. Downstream visualization can then use this shared timestamp to align a given audio event with the correct rendered frame, delayed only by the ring buffer and the audio thread's own buffer latency.

2.4.3 Computational Budget

As mentioned in 2.4, a budget of 23 ms for the callback of the extractors is the limit. Since the FFT-dependent methods, like STFT, chroma and Onset Detection, run in $O(N \log N)$ time and RMS runs in $O(N)$ time, an empirical check is necessary to confirm the chosen configuration actually stays within this budget in practice, rather than assuming it from complexity alone. Section 5.3 reports this empirical validation.

2.5 Visual Feedback in Music Learning

A human has two separate information processing systems. For the visual material and for the auditory material. So combining those visual and audio feedback can work together. While watching a text on screen giving us visual feedback, the brain subsequently convert it mentally into sound for processing in the auditory channel [11, p. 102]. Based on those findings, it can be assumed that it works as well with audio-based visual feedback. But humans only have a limited capacity of the information they can process [18, p. 261]. They are even limited to how much one channel at one time can comprehend. So if there is too much happening, it can be an overload to the channel and less information will be processed. Design choices for how a sound is presented visually are quite important [11, p. 102]. So to decide what kind of feedback

appears, the include of Blanco's claim that immediacy, granularity and modality trade-offs play a role in comprehending the visual in an effective way is necessary. The immediacy of the external feedback or concurrent knowledge of the result, leads to a more effective way of learning. But not all visuals are equal, if the real-time feedback does not have the knowledge of result, it results in no improvement of learning efficiency in pitch-matching for children. A modality trade off is, that it requires visual attention, which leads to worse results for some learners. However, testers also reported that auditory feedback alone did not give them enough information. They could tell a note was wrong, but not whether they had guessed too high or too low, so they did not know which way to correct it. So a visual feedback would have improved their sense of correction [2, p. 2].

3 Concept for the Audio Visualization System

In this section the layout and the concept for the audio visualization system is described.

3.1 Requirements

This section outlines the requirements for the system, which are split into functional and non-functional requirements.

3.1.1 Functional requirements

The system captures the audio output from an audio source in real time with the audio thread. The captured samples must be safely directed to the main thread without losing audio data. The methods like RMS, FFT etc. all have the same base raw PCM input, so the evaluation is based on a neutral test experience. The methods must be swappable without the need to modify any other components. The visuals are driven and triggered by the buffered audio parameters. The visual system is not triggered by the play function of the tap of a note on the main thread, it starts as soon as the audio thread captures the audio data and the analysis methods detect a note onset. The visuals always return to a idle state. The methods RMS, FFT, Chroma, STFT and Onset Detection need to be implemented. The data captured gets written in a CSV file for empirical evaluation purposes.

3.1.2 Non-functional requirements

The system must process each buffer within the 23ms real-time budget which was established in 2.4. This implies as well that no crashes or audio dropouts are occurring due to cross-thread access. The synchronization of audio and visual via `dspTime` is staying within a tolerable latency bound. The system is easily extensible and maintainable. Common interfaces for extraction methods and for visualizations are making it faster to implement new ones. The visualization is clear and understandable for beginners and non-experts in music learning.

3.2 Selection of Analysis Methods

Since the research question was which audio analysis methods are suitable for the real-time visualization of musical parameters in interactive music-learning applications, considering performance, interpretability and user perception, the decision was made to go with five different approaches to analyse the audio data. As established in 2.4.3, RMS's $O(N)$ cost and the shared $O(N \log N)$ cost of the FFT-dependent methods mean all five are theoretically fitting the real-time budget, subject to the empirical verification in Chapter 5.3.

- RMS
- FFT
- STFT

- Chroma feature
- Onset Detection

Each of them are targeting different kinds of attributes of an audio wave which then can be mapped on different visual parameters to drive them.

- energy
- static frequency content
- time-varying frequency content
- pitch class
- events

Together they cover the parameter space for the research question.
For energy: RMS vs. simpler peak-amplitude tracking or more complex loudness models (e.g., loudness units relative to full scale(LUFS)-style perceptual loudness [19]). The RMS sits in between both of them. It is cheaper and simpler than perceptual loudness models and more representative of actual signal energy than a bare peak value.
For static frequency content: FFT vs. autocorrelation-based pitch estimation or the Goertzel algorithm, which targets single frequencies [8]. FFT gives the full spectrum rather than a single pitch estimate, which matters since pitch alone is not the only target here. The general spectral content is better for visualization.
For time-varying spectral content: STFT vs. the CQT [3] or wavelet transforms. STFT is simpler to implement with a fixed real-time window and hop size and is the standard choice in real-time contexts. CQT and wavelets are more accurate, but more expensive and less common outside offline analysis.
For pitch class: chroma vs. direct fundamental-frequency estimation like autocorrelation or YIN [4]. Direct fundamental-frequency estimation is more precise, but more failure-prone on noisy or polyphonic signal content, while chroma's octave-folding makes it robust to those errors at the cost of losing register information. Which is an acceptable trade-off since the note identity is the only important aspect, not exact pitch height, for the mapping that is being done.
For events: spectral flux vs. simple amplitude-threshold triggering or complex-domain or phase-based onset detection [1]. Amplitude thresholding misses onsets where energy does not change much, even though the spectral content does.
Phase-based methods are more accurate for soft onsets, but are unnecessary overhead given the marimba which is the instrument the app uses as a base. Marimba strikes have extremely sharp, clean transients.
But since marimba is not our only instrument available in the app to represent a note, it is important that the methods are robust against timbre. Figure 3.1 illustrates this using three instrument presets played at the same pitch (C4). Panel (a) shows that although the fundamental frequency is identical at around 261 Hz, the relative strength of the harmonics

differs substantially between the three instruments. The default instrument concentrates nearly all of its energy in the fundamental frequency, the organ has a perceptible third harmonic and the bell distributes its energy across many detuned partials. A frequency-domain method that is adjusted to one of these harmonic distributions, such as picking the single largest spectral peak or matching a fixed harmonic template, would misjudge instruments with a different distribution. Panel (b) illustrates the corresponding envelopes with a fixed onset threshold at 20 % of the peak. The bell crosses the threshold almost instantly, around 1 ms, while the default instrument and the organ take considerably longer, around 25 ms and 30 ms respectively, because their energy builds up more gradually instead of starting with a sharp transient like the bell. A fixed-threshold onset detector calibrated on a sharp, percussive transient would therefore mistime or miss onsets on instruments whose energy rises more slowly. Overall, these results show that neither spectral shape nor onset timing is consistent across timbres, so a method that only performs well for one instrument's characteristics is not reliable across the range of sounds the app needs to handle. This is why the selection above favors methods, such as chroma folding and spectral-flux-based onset detection, that are robust to these instrument-dependent differences rather than approaches tuned to a single instrument.

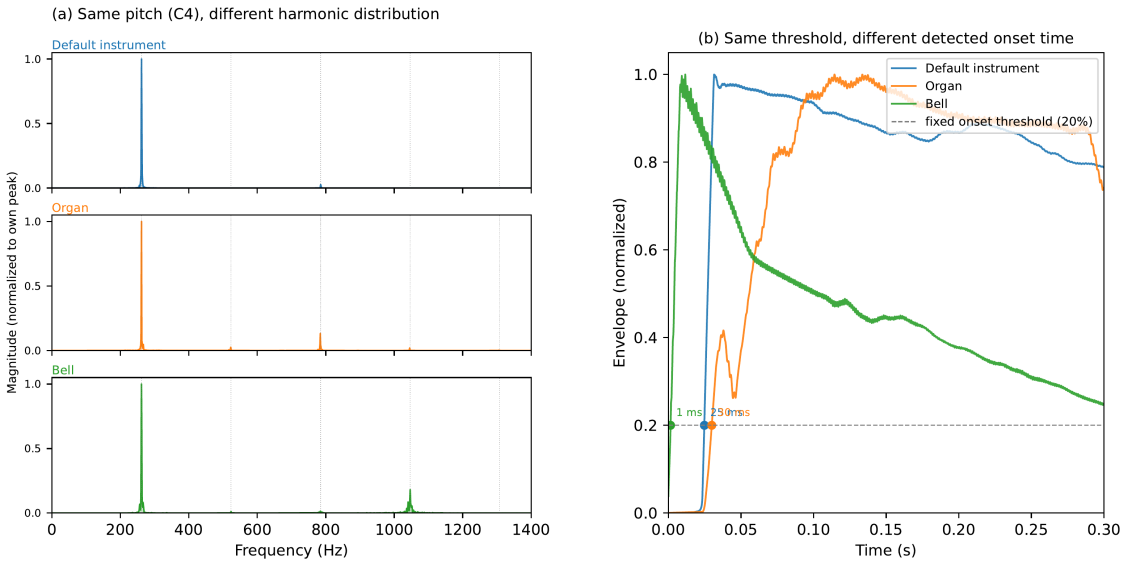


Figure 3.1: Timbre-dependent variability across three instrument presets played at the same pitch (C4). (a) Magnitude spectra normalized to each instrument's own peak, showing different harmonic distributions. (b) Envelopes with a fixed onset threshold (20 %), showing that detected onset time varies from about 1 ms (bell) to 25–30 ms (default instrument and organ). Own measurement.

With the choice and robustness of these five methods established, the following section describes how they are integrated into a system architecture that can capture, process and visualize the audio in real time.

3.3 System Architecture

As a framework and base for these tests and implementation is the wordless, minimalistic music learning app Tone, an app programmed in the Unity engine by Willi Schorrig [17], which is based on cognitive load theory [18], flow theory [5] and minimalistic design principles [14]. The tone app generates `ToneBlock_v2` which needs to be placed correctly. When tapped, a sound or tone is played and the visualization are supporting the identification of the pitch and make a coherent experience in the pitch-matching puzzle mode. But overall, the visuals make the experience with the app more enjoyable and make sound comprehensible. The architecture for the audio-visualization system can be split into the Audio Capture, Feature Extraction, Mapping Layer and Visualization as well as Evaluation Logging. Figure 3.2 shows this pipeline. The structure for the system is layered with interfaces so it is more easily extensible. So to not overload the audio thread and to stay within the real-time constraints, as established in 2.4.1, it only captures the sound data and hands it off to the main thread for heavier computations like the FFT-based methods.

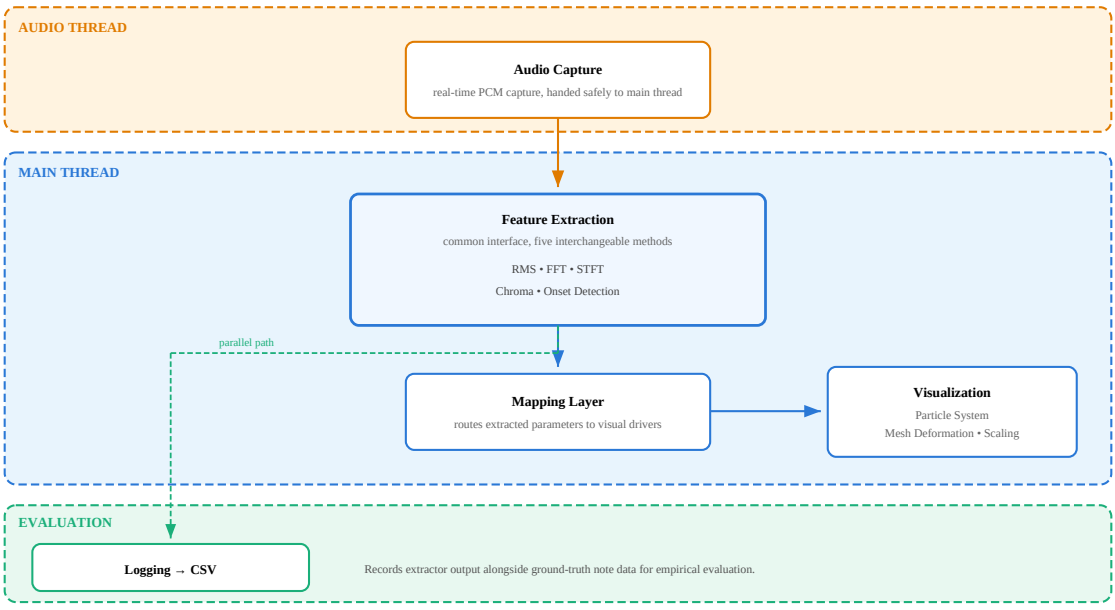


Figure 3.2: Pipeline of the system architecture. The audio thread captures the audio data and sends it to the main thread for processing and visualization. The main thread handles feature extraction, mapping and rendering of visualizations.

3.3.1 Audio Capture

As established in 3.3, the audio source is the `ToneBlock_v2` from `MusicPhraseGenerator`. The `MusicPhraseGenerator` is a script from the Tone app which generates customizable tones for the user to distinguish and correctly place. The system is capturing raw audio in real time from Unity's audio pipeline. Since the strict timing budget which was determined in 2.4.1, the double-buffering mechanism is a safe way to hand that data from the audio thread, on

which the capturing occurs, to the main thread, without risking a torn read or blocking the audio thread. One buffer is written by the audio thread, then gets locked for a short moment and copied into a second buffer which the main thread can then safely read. The lock duration needs to be kept at a minimum so it does not cause an audible dropout.

3.3.2 Feature Extraction

Once the main thread gets the second buffer with the audio data it gets passed to the Feature Extraction stage. To build a coherent system where every extraction method can be swappable and the system is easily extensible, a single shared interface is necessary. Every method targets a different musical parameter, which was explained in 3.2. They share one interface, so adding a sixth method later would mean that it is only necessary to write one new component and not modify the whole pipeline around it. The output of all five methods gets collected into a single shared representation. It is a synchronized snapshot per audio buffer, tagged with a timestamp which was established in 2.4.2. It contains everything the system currently knows about that sound at that instant. This shared collection is the handoff point between analysis and everything downstream and is intentionally read-only from that point on. Nothing can flow back upstream and corrupt it. The shared representation is then passed to the Mapping Layer, which is responsible for translating the musical parameters into visual parameters. This shared representation is also written to a CSV file for empirical evaluation purposes, which is described in the next section.

3.3.3 EvaluationLogging

Whenever a note is played, its audio data must be logged for evaluation purposes. The `EvaluationLogger` component is responsible for this and is implemented as a per-instance component, one running per active-playing note, which reports to a singleton `EvaluationSession` that owns the CSV file and the tap counter.
The evaluation logging system separates into two parts: network-facing session logging and local-only evaluation logging. Network-facing session logging writes the session data required for the user survey (5.4), while local-only evaluation logging processes each methods output for every active-playing note and writes it to the `EvaluationSession`. This separation ensures that local, exploratory technical testing never pollutes the network-facing survey data and that concurrent taps do not corrupt each other's rows in the shared CSV file.
For every completed tap, a single row is written containing:

- note identity and duration;
- peak RMS and envelope;
- instrument index, clip peak amplitude and the derived `drive` value (envelope normalized to the clip's peak);
- onset ever-detected flag, latency and strength;

- STFT mid/high band energy;
- sampled snapshots of FFT dominant frequency, RMS and the chroma vector at fixed offsets post-onset (frames 1, 3, 10, 20, 30), capturing detection behaviour across the note's decay rather than only at its peak.

These fields are used in 5.3 to assess each methods detection accuracy and robustness to instrument timbre. In addition to the per-tap summary described above, `EvaluationLogger` supports a time-series capture mode for a single tap. Rather than aggregating peak values into one row, it writes one row per audio frame, containing the full STFT magnitude spectrum, chroma vector, RMS, envelope, dominant frequency and onset state at that instant. This mode is used to produce fine-grained within-note figures (e.g. spectrograms, spectral centroid decay) that the per-tap summary cannot represent. With the audio features captured, the next step is translating each one into the shared visual vocabulary, like particle size, speed, emission rate, lifetime and color that the game actually renders.

3.4 Mapping Musical Parameter

3.4.1 Audio Features

The five analysis methods established in Section 2.3 each output a different kind of musical information: energy, static frequency content, pitch class, event timing and time-varying spectral content. To keep the comparison between methods as controlled as possible, each method is paired with exactly one dedicated `IVisualResponder` implementation, which maps its output onto the same shared visual vocabulary (particle size, emission rate, color hue, lifetime and outward speed of a `ParticleSystem`). Differences in the resulting visuals can then be attributed to the analysis method itself, rather than to different mapping choices between methods.

3.4.1.1 Frequency

`FFTResponder` maps the dominant frequency bin, the bin with the largest magnitude in the Hann-windowed FFT spectrum, onto color hue, linearly interpolated between a fixed low and high frequency bound (`MIN_HZ`–`MAX_HZ`). Since the dominant bin can fluctuate between adjacent harmonics from one frame to the next, the hue is locked a few frames after note onset rather than recomputed every frame. The shared envelope value drives size and emission rate the same way it does for the other responders, so hue is the only variable specific to this method.

3.4.1.2 Amplitude

`RmsResponder` maps the RMS/envelope value directly onto particle size and emission rate, while color is held at a fixed neutral white regardless of which note is playing. This is a

deliberate negative case, since RMS cannot carry any frequency information at all, the mapping is restricted to only the two visual dimensions RMS output can genuinely justify, how much energy is present and when, rather than inventing a pitch mapping that the underlying analysis does not support. This makes the resulting visualization useful as a baseline for isolating what loudness-only feedback looks like.

3.4.1.3 Onset

`OnsetResponder` fires a fixed-ratio burst on size and outward speed on the exact frame `onsetDetected` is true and otherwise lets `onsetStrength`, the continuous spectral-flux value, decay the burst back down gradually. Onset detection only tells*when* something started, not *what* it is, so, similar to `RmsResponder`, color is fixed to white and the mapping is scoped to timing-related visual dimensions only.

3.4.1.4 Chroma

`ChromaResponder` maps the ground-truth symbolic pitch class onto a 12-step hue wheel

$$\text{hue} = \frac{\texttt{pitchClass} \bmod 12}{12}$$

so even pitch classes which are an octave apart share the same color. That makes it consistent with the pitch-class equivalence established in Section 2.1.1 and Figure 2.1. A secondary mapping uses the chroma vector's largest bin value to drive particle lifetime. Since `ChromaExtractor` already normalizes the chroma vector so its largest bin is 1.0 whenever any signal is present, this value does not actually vary with how ambiguous the pitch-class detection was. In practice, it functions as a presence/silence signal: it allows particles to live longer while audio is playing and die off quickly once it stops.

3.4.1.5 Spectral Density

`STFTResponder` splits each frame's magnitude spectrum into three bands (bass, mid and high) using fixed frequency cutoffs, then normalizes each band's average magnitude against the frame's own RMS value. After that, it smooths each band over time with an asymmetric attack/release filter to avoid frame-to-frame flickering. Bass energy drives size, mid energy drives emission rate and high-frequency energy shifts hue slightly away from the base pitch-class color. This lets a single note's visualization change in real time as its spectral balance evolves, e.g., a bright attack settling into a duller sustain. That is only possible for STFT, since the other four methods are each limited to a single scalar or a static pitch class per frame.
Now comes the final stage of the pipeline, where the mapped visual parameters are actually rendered on screen.

3.5 Visualization Concepts

Since time constraints favored depth on a single visualization approach across all six responders rather than breadth across several shallow ones, the system implements a single

`ParticleSystem`-based visualization, shared by every responder.

3.5.1 Particle System

All six responders (the five single-method ones plus `CombinedResponder`, Section 4.3) drive a single shared `ParticleSystem` per `ToneBlock`, using the same Unity particle parameters as a common visual vocabulary: `startSize` (radius), `startSpeed` (how fast particles move outward), `emission.rateOverTime` (density), `startLifetime` (how long a particle persists) and per-instance material color. Constraining every method to those parameters, rather than letting them invent their own visual language, resulted in an easy-to-compare system. Any visual difference between two responders reflects a difference in the underlying analysis output, not a difference in how expressively each was allowed to render.
From here on comes the implementation of the system architecture and the evaluation of the methods in Chapter 4 and 5, respectively.

4 Implementation

4.1 Unity Architecture

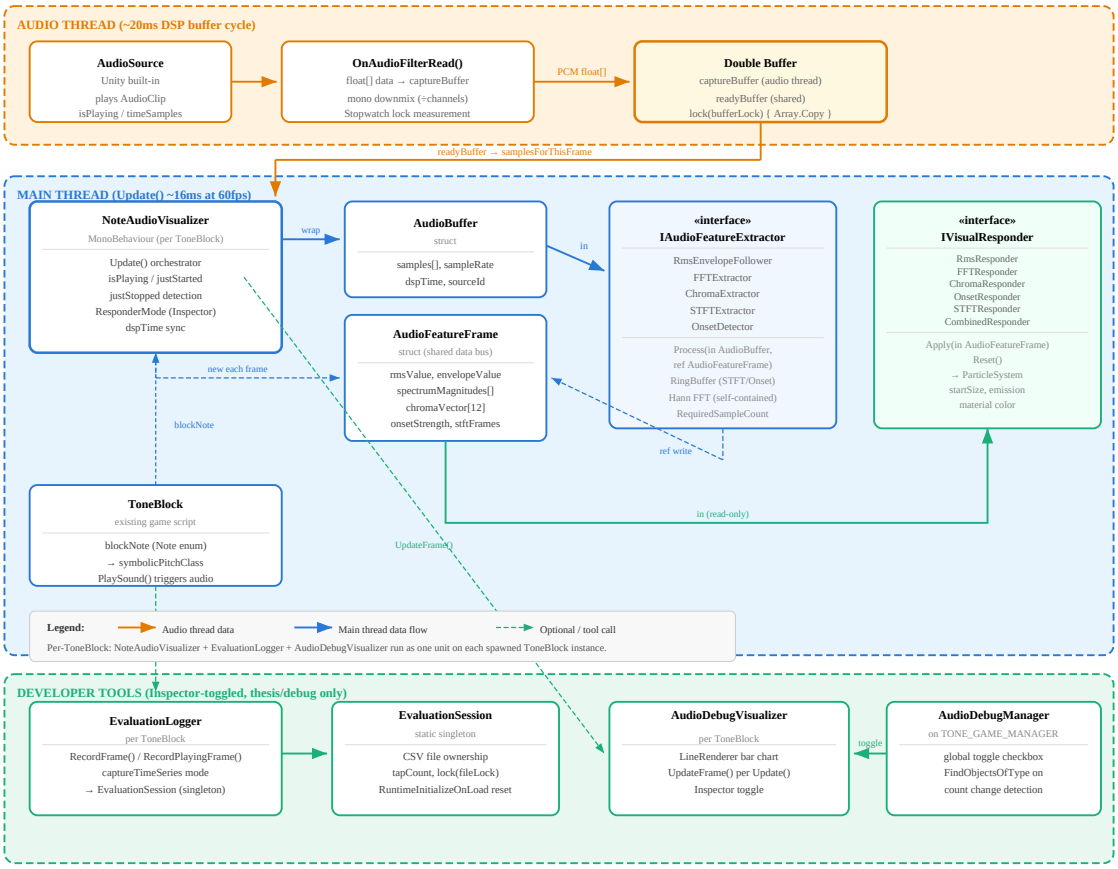


Figure 4.1: UML diagram of the system architecture.

The architecture separates audio capture, feature extraction and visual response and developer/study tooling into three layers, shown in Figure 4.1. The following subsections describe the components that make up the main (middle) layer and the design decisions behind them.

4.1.1 Component Structure

`NoteAudioVisualizer` attaches alongside the existing `ToneBlock` component on the same GameObject (`[RequireComponent(typeof(AudioSource))]`). Then `NoteAudioVisualizer` listens to whatever that `AudioSource` actually outputs, rather than reacting to the moment `ToneBlock.PlaySound()` is called. This solves the timing-desync problem between "when `Play()` was invoked" and "when audio is actually audible" that an earlier version of the pipeline had and it touches none of `ToneBlock`'s own trigger logic.

4.1.2 Shared Data Contracts

Every audio extractor receives the identical `AudioBuffer` (raw mono-summed PCM, sample rate, DSP timestamp) and writes its results into a shared `AudioFeatureFrame` struct. Keeping them both as the single source of truth is what makes the five analysis methods comparable at all. They all read the exact same samples, so any difference in their output is attributable to the analysis method itself, not to different input data, which is the same fairness argument already made for the visual mapping side in Section 4.3, here stated for the analysis side.

4.1.3 Extractor and Responder Registration

`Awake()` unconditionally registers all five `IAudioFeatureExtractor` implementations. The execution order does not matter, since each only reads the shared buffer and writes its own fields. The Responder selection works differently. Exactly one `IVisualResponder` is instantiated, chosen once at `Awake()` by a switch on the Inspector-set `responderMode` enum, which has six cases, which are the five single-method responders plus `CombinedResponder`. This is the concrete mechanism behind the claim made in Section 4.3 that extractor execution is independent of which responder is visually active. All five methods outputs are always computed into `AudioFeatureFrame` regardless of `responderMode`. Only the one driving the particle system changes. Changing `responderMode` in the Inspector never requires a recompile.

4.1.4 Note Lifecycle Detection

Because the visualizer reacts to the `AudioSource` rather than to an explicit play call, it has to notice the start and stop of a note itself from the variables `isPlaying` and `timeSamples`. This includes distinguishing a clean start from a retrigger, the same note tapped again while it is already playing. A retrigger is detected when the note was already playing last frame (`isPlaying` true both this frame and last) and `timeSamples` has just dropped back below its value from the previous frame, rather than continuing to advance. A minimum sample threshold guards against falsely detecting a retrigger on the very first frame, when both values start at zero. This distinction matters because `isPlaying` never drops back to `false` during a retrigger. Relying on its False-to-True transition alone would miss every retrigger and leave `noteJustStarted` stuck at false on rapid repeated taps.

4.1.5 Shared Per-Clip Analysis Cache

`CombinedResponder`'s normalization (Section 4.3) needs each clip's peak amplitude and decay-tail start point. Rather than recomputing these from a fresh `AudioClip.GetData()` scan on every single note play, they are computed once per distinct `AudioClip` and cached in a static dictionary shared across every `NoteAudioVisualizer` instance in the scene. Since the same handful of clips (one per note/instrument combination) gets replayed constantly across many `ToneBlocks`, this avoids repeating a full-waveform scan hundreds of times over a session. The

next step is to delve into the actual audio analysis methods themselves, which are the core of the research question.

4.2 Audio Analysis

This section describes what each method *computes* from the raw audio. Every extractor implements the same two-member contract, `Process(in AudioBuffer, ref AudioFeatureFrame)` and `RequiredSampleCount`, so the pipeline can skip calling an extractor on a buffer too short to be meaningful. Implementations are intentionally stateless where possible, so no `MonoBehaviour` and no `GameObject` reference which allows extractors to be unit-tested and benchmarked in isolation. This is relevant to the performance evaluation in Section 5.3.

4.2.1 RMS/Envelope

`RMS = sqrt(mean(samples`^{2}`))` is computed over the buffer, needing only 512 samples to be meaningful, which is far fewer than the FFT-based methods. They need a full window for frequency resolution. A one-pole smoothing filter with separate attack (0.9) and release (0.3) coefficients turns the raw per-buffer RMS into the continuous `envelopeValue` that responders actually use, rising quickly on a transient and falling more slowly on decay to avoid frame-to-frame flicker.

4.2.2 FFT

A radix-2 Cooley–Tukey FFT is computed over a Hann-windowed buffer, implemented from scratch rather than using Unity's built-in `GetSpectrumData` so that every extractor windows and transforms the same raw samples. A fairness requirement for comparing methods. The extractor uses the visualizer's full configured window (2048 samples by default, ≈21.5 Hz bin resolution at 44.1 kHz), taken from the most recent samples in the buffer. The same spectrum yields two outputs. One is the dominant bin (highest magnitude, which feeds `FFTResponder`'s color mapping and the dominant-frequency figures in Section 5.3) and the other is the spectral centroid (magnitude-weighted mean frequency, which feeds the centroid-decay figure in Section 5.3), both computed in the same pass over the same bins.

4.2.3 Chroma

Chroma extraction builds on the same FFT machinery and the same 2048-sample window described above, then folds the magnitude spectrum's bins into 12 pitch classes via frequency → semitone → pitch class, with a global correction offset (`PITCH_CORRECTION_SEMITONES`) discussed further in Section 5.3.

4.2.4 Onset

Onset detection uses spectral flux which is the sum of positive-only magnitude differences between consecutive hops, so that energy appearing registers as an onset. Meaning a note

ending does not itself trigger a false onset. The extractor is self-contained, with its own internal ring buffer and its own independent FFT and deliberately uses a smaller window (1024 samples, 512-sample hop) than FFT/Chroma's 2048. The reason is onset detection needs several hops of temporal context within a single `Update()` callback rather than one high-resolution snapshot. A fixed flux threshold (0.03) combined with a 10-hop cooldown prevents a single physical strike's transient from registering as multiple onsets. The continuous `onsetStrength` signal is separately smoothed with its own exponential decay (0.85 per frame) so responders have something gradual to map even between discrete onset events.

4.2.5 STFT

The STFT extractor uses the same 1024-sample window and 512-sample hop as onset detection (also self-contained, with its own internal FFT), but retains a rolling 8-frame history rather than immediately collapsing each hop to a single flux number. It is producing the time × frequency matrix that the spectrogram figure in Section 5.3 and `STFTResponder` and `CombinedResponder`'s band-energy mapping both depend on. Because a single `Update()` callback can span more than one hop, the extractor tracks which hop produced the most energy that callback and returns that hop's full spectrogram snapshot. Otherwise a buffer that happened to end on a quiet hop could report a near-silent spectrogram despite a loud onset a few samples earlier in the same callback.

4.3 Mapping System

4.3.1 Responder Interface and Selection

All six responders implement the same `IVisualResponder` interface, `Reset()` and `Apply(in AudioFeatureFrame)`. `NoteAudioVisualizer` instantiates exactly one responder per `ToneBlock`, chosen once at `Awake()` from the Inspector-set `responderMode` enum. Only the rendering choice is made this way. All five extractors keep running every frame regardless of which responder is active, so switching `responderMode` changes only what gets visualized, never what gets analyzed or logged (Section 3.3.3). The five single-method responders are covered alongside their analysis methods in Section 3.4. This section covers the sixth, `CombinedResponder`.

4.3.2 Motivation

The five single-method responders exist to keep each analysis method isolated for comparison, per the research question. That isolation is exactly what makes them unsuitable for the app's actual gameplay/study use on its own. A user playing through real levels hears multiple instruments (marimba, guitar, bass, percussion) and a single-extractor responder ,driven by raw envelope amplitude, reproduces the timbre problem shown in Section 3.2 in the visuals. A loud, sample guitar note fills most of the screen, while a quieter marimba note at identical perceived intensity barely registers. `CombinedResponder` was built specifically for this practical,

study-facing use, fusing signals from multiple methods rather than relying on any one of them alone. The five single-method responders remain fully intact and selectable in the Inspector for isolated testing. The `CombinedResponder` is additive, not a replacement.

4.3.3 Design

Color comes directly from the ground-truth symbolic pitch class the game already knows it is playing (the same 12-tone hue wheel as `ChromaResponder`), rather than from a detected value. Always correct regardless of analysis noise, since correctness of the note-to-color mapping is not itself part of what this responder is testing.

Size, density and speed are driven by a normalized "drive" value, which is the current envelope divided by that specific note's own peak amplitude (`clipPeakAmplitude`), rather than by raw envelope amplitude. Dividing by each clip's own peak maps every instrument's dynamic range onto a common 0–1 scale in principle. In practice the cross-instrument spread narrows, but does not vanish (see Section 5.3). This reduces rather than eliminates the timbre-driven loudness gap, while each instrument still decays at its own natural rate afterward (marimba quickly, guitar more slowly), preserving the audible difference the visualization should carry, rather than an artifact of sample loudness.

Extra density comes from the same mid and high spectral-balance ratio `STFTResponder` uses, layered on top of the amplitude-driven density.

Onset bursts apply a fixed-ratio kick to size and speed on the exact frame an onset fires, independent of the drive value, so an instrument with a soft, gradual envelope still produces a visually distinct attack moment the way a percussive one naturally does.

Pitch drives an additional, continuous adjustment to size and speed on top of loudness. The lower notes render larger and slower, higher notes smaller and faster, scaled per semitone from a configurable reference point rather than normalized against the full theoretical pitch range. This was a deliberate correction after an early version normalized against the full ≈5-octave range and found that real test phrases, which typically span only a handful of semitones, produced barely visible size differences as a result. The normalizing against a range far wider than any single phrase actually uses diluted the very differences it was meant to show. Color still wraps every octave, consistent with the pitch-class equivalence established in Section 3.2.The size and speed deliberately do not, so that, e.g. C1 reads as smaller than B0 rather than jumping back to "biggest" on crossing into a new octave.

A note's audio has a quiet decay tail after its audible content ends. A live per-frame amplitude threshold alone cannot reliably distinguish "still genuinely decaying" from "now in the dead tail", since both sit at a similarly low envelope level. This is resolved with a ground-truth check. Each clip is scanned once, on first use, to find the last point at which it ever rises back above roughly 5% of its own peak again. Once playback passes that point, the tail is known with certainty rather than inferred. Existing particles are then explicitly cleared (`ParticleSystemStopBehavior.StopEmittingAndClear`) rather than left to their own scheduled lifetime. Since simply stopping new emission was not sufficient, particles already

spawned during the loud part of a note can carry lifetimes of over a second and continue rendering well after the note has ended otherwise.

4.3.4 Scope Note

`CombinedResponder`'s per-note loudness normalization is computed from a one-time offline read of each `AudioClip`'s own waveform data (`AudioClip.GetData()`), not from the real-time analysis methods under study. This is a deliberate, scoped exception. It is used only in the survey-facing responder, never in the five single-method responders the research question directly compares. It exists because a purely real-time equivalent was out of scope for this thesis. The real-time alternative is named explicitly as future work in Section 6.

4.4 Real-Time Visualization

4.4.1 Audio-Thread to Main-Thread Handoff

`OnAudioFilterRead()` runs on Unity's audio thread and accessing or using other GameObjects or Components from inside of it is unsafe. It only does the minimum and mono-sums the incoming audio samples and copies them into a buffer. This happens under a lock scoped only to that copy, not the rest of the callback which is the double-buffer swap. `Update()`, which runs on the main thread on each rendered frame, works with the last-completed buffer under the same lock and builds an `AudioBuffer`. Only after that it runs the extractor/responder pipeline. This way, every Unity API call stays safely on the main thread while still capturing every audio callback's data. Lock duration gets measured with a `Stopwatch`, which oversees critical sections and logs the average and max duration. This always confirms that the transition stays in the microsecond range, which is measured directly in Section 5.3).

4.4.2 Per-Frame Update Cycle

For a playing note, each `Update()` call builds a fresh `AudioFeatureFrame`, runs it through all five extractors, applies it to the active responder and forwards it to `EvaluationLogger`. All of it within the same frame, so the particle system is never more than one `Update()` behind the audio it represents. Idle `ToneBlocks` skip this pipeline entirely rather than running it on every note object in the scene regardless of state, since `OnAudioFilterRead()` keeps firing even during silence for any enabled `AudioSource`.

4.4.3 Avoiding Visual Jitter

Several extractors and responders apply their own smoothing specifically because a raw per-frame analysis value recomputed every 10–20 ms is noisier than what looks good rendered directly. RMS's one-pole attack and release envelope, onset's exponentially decaying strength signal and `STFTResponder` and `CombinedResponder`'s own attack and release filtering on band energy all exist for smoothing the visual signal.

4.5 Performance Optimization

4.5.1 Deliberate Optimizations Already in Place

- **Idle skip.** The full extractor/responder pipeline never runs for a `ToneBlock` that is not currently playing, even though its `AudioSource` keeps calling `OnAudioFilterRead()` regardless.
- **Per-clip analysis caching.** Each `AudioClip`'s peak amplitude and tail-start point are computed once (one `GetData()` scan) and cached across every `NoteAudioVisualizer` instance, rather than re-scanning the same clip's full waveform on every note play.
- **Pre-allocated working buffers.** Hann window coefficients and FFT real and imaginary arrays are allocated once at construction and reused on every `Process()` call in the FFT, chroma, onset and STFT extractors, rather than allocated fresh per frame.
- **Minimal-scope locking.** The pipeline's only lock guards a single buffer swap between the audio and main thread, measured in practice to stay in the microsecond range.

4.5.2 Remaining Costs Worth Naming Explicitly

- **Redundant FFTs.** FFT, chroma, onset and STFT each compute their own independent windowed FFT from largely the same input every update, rather than sharing one computed spectrum across extractors. A direct trade of computation for the isolation and swappability the `IAudioFeatureExtractor` design is built around. This was not measured directly. Given the idle-skip optimization (Section~4.5.1) and the app's tap-driven, largely monophonic note density, the redundant computation is treated as an accepted design cost rather than a profiled one.
- **Per-frame allocations that survived the pre-allocation pass.** `FFTExtractor.Process()` allocates a fresh `magnitudes` array and `ChromaExtractor.Process()` allocates a fresh 12-element `chroma` array on every call, unlike the buffers above. It is small individually, but real per-frame garbage multiplied by however many notes are playing at once.
- **Full-buffer shifting in STFT.** The rolling spectrogram is maintained by shifting every retained frame back one slot on every hop ($O(\text{numFrames} \times \text{numBins})$ per hop) rather than a circular index – simple and correct, not the cheapest possible approach.

4.5.3 Sampled vs. Full-Resolution Logging

`EvaluationLogger`'s default per-tap summary samples only five fixed post-onset frames (Section 3.3.3) rather than logging every frame of every note, specifically to keep per-session CSV size and disk input and output manageable across a full data-collection pass. The separate time-series mode that captures every frame of a single tap (used for the spectrogram and centroid-decay figures in Section 5.3) is deliberately reserved for one tap at a time rather than run continuously, for the same reason.

5 Evaluation

5.1 Experimental Setup

The study used a within-subjects, counterbalanced design comparing two visual conditions: the existing static visual system already in Tone ("Old") and the new particle-based system built for this thesis ("New"), driven by `CombinedResponder` during normal play (Section 4.3). The condition alternated every level a participant played, so each participant experienced both conditions across the session rather than only one. The starting condition was assigned pseudo-randomly via participant ID parity. The even-numbered participants started on the Old condition and the odd-numbered participants started on New, which intended to approximate an even split of starting conditions across the sample. The realized split is reported in Section 5.4, since parity-based assignment on a randomly generated ID is not a deterministic guarantee for any given sample size.

The experiment took roughly 10–15 minutes to play, plus a further 5 minutes for the questionnaire. Based on this length, the expected participant count was around 15–20 people. Participants ranged from non-musicians to experienced players. Inexperienced participants are a good measure of how well the visual system supports note identification on its own, without prior musical training to fall back on. The experienced participants' musical background provides a check on how accurately the visuals actually track the underlying audio. Testing was limited to Android devices and Windows and both remote and in-person sessions were possible. The test had six levels of five phrases each, with the instrument for each phrase fixed in advance. Level one to four rotated between marimba and guitar phrases. Level five and six mixed marimba and guitar phrases within the same level, so that participants were more likely to notice the visual system responding to a timbre change mid-level rather than only between levels.

After the last level, participants completed a post-session questionnaire covering two core research dimensions, Understandability and Perceived Synchronicity, plus three supplementary dimensions collected for broader qualitative context: Musical Coherence, Visual Appeal and Overall User Experience. In every question, participants compared the old condition against the new condition, labeled A and B respectively rather than "Old" and "New", which was a deliberate design choice so participants would not be biased by knowing which version was new and could approach the comparison with no preconceptions. Participants were never told the purpose of this thesis, so that they could focus on the app itself, letting the questionnaire show whether the visuals were noticed at all and how much they affected participants' experience independent of that context. Participants were always shown a side-by-side comparison image of both versions for reference (Figure 5.1).

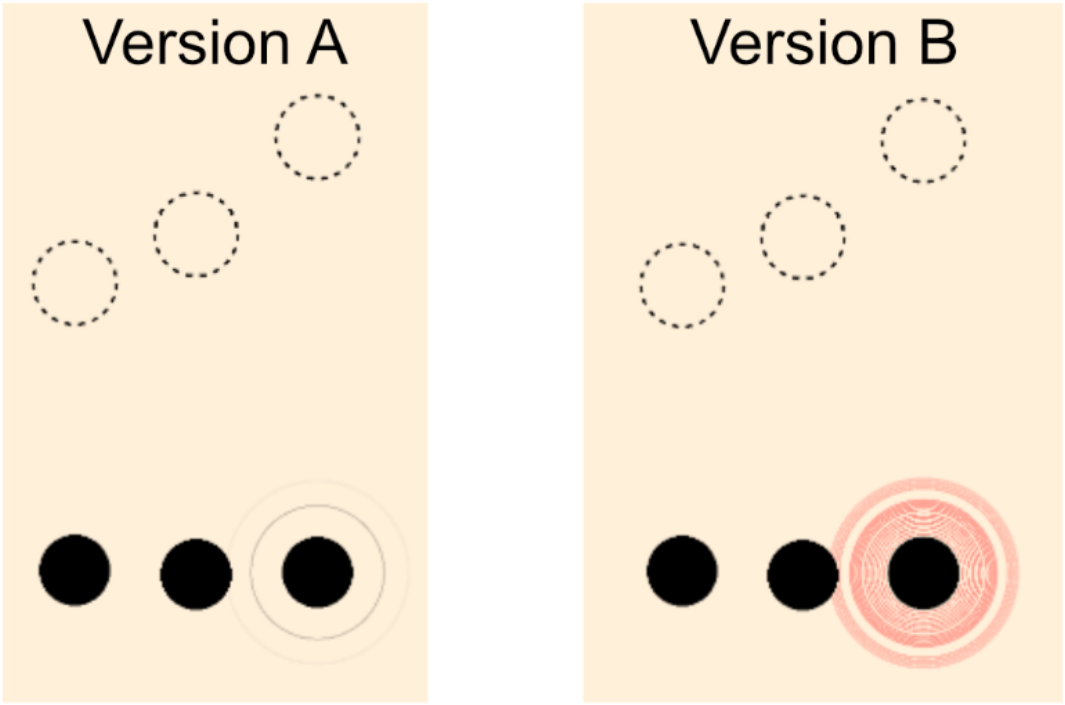


Figure 5.1: Side-by-side comparison of Version A (Old) and Version B (New) shown to participants in the questionnaire.

Figures 5.2 and 5.3 show the two conditions in sequence across four consecutive note taps, illustrating the Old system's static response compared to the New system's per-note visual change.

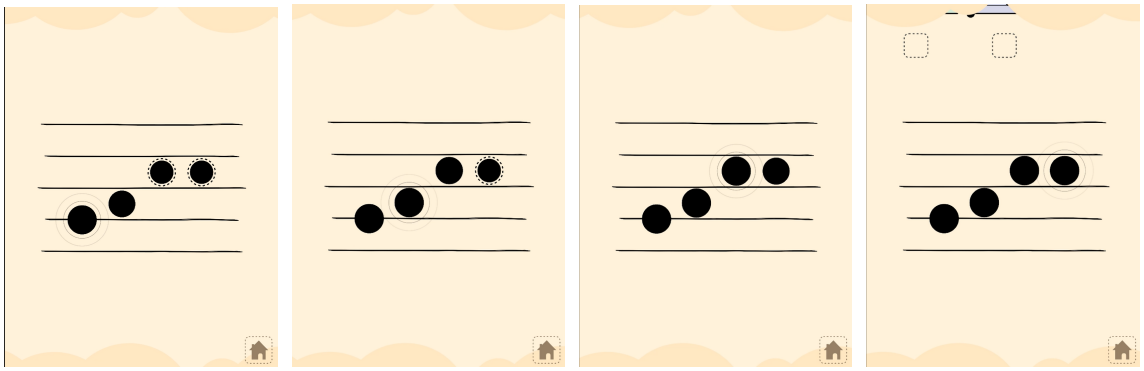


Figure 5.2: Old (static) visual system responding to four consecutive note taps.

Participants were then asked a multiple-choice question about their prior music education or instrument experience, with the options "None", "Some", and "Extensive", to allow separating beginner responses from experienced users responses during analysis. For the next three sections, the answer scale ranged from "Much prefer A", "Slightly prefer A", "No preference", "Slightly prefer B" to "Much prefer B."
The Understandability section held the questions:

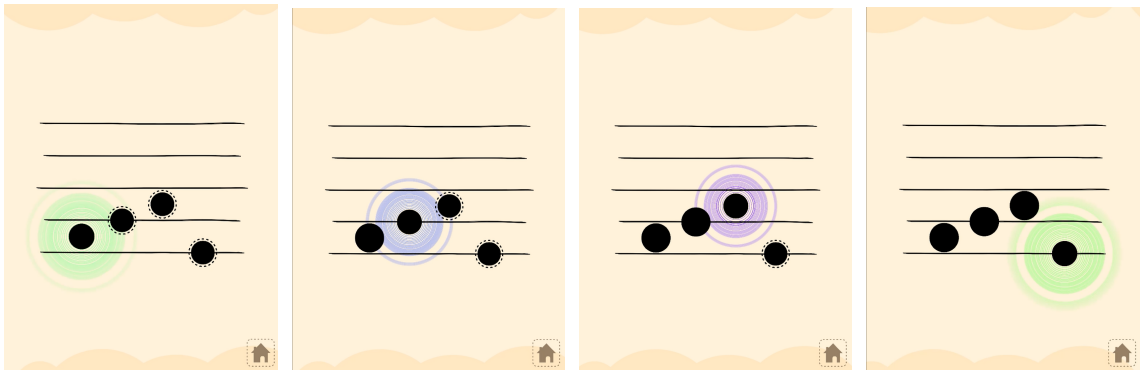


Figure 5.3: New (particle-based) visual system responding to the same four consecutive note taps.

- Which version made it easier to tell which note was being played?
- Which version made it easier to tell how long a note lasted?

The Perceived Synchronicity section held the questions:

- Which version's visuals felt more in sync with the sound, with less noticeable delay?
- Which version's visuals felt more like they stopped when the sound stopped, rather than lingering afterward?

The Musical Coherence, Visual Appeal and Overall User Experience section held:

- Which version's visual changes (bigger/smaller, faster/slower) matched what you'd expect from the pitch and loudness of the note better?
- Which version did you find more enjoyable to look at?
- Overall, which version did you prefer?

The last section, of only secondary importance to the research question, was Color of Version B, where participants rated statements about the New system's color changes on a 5-point Likert scale from agree to disagree:

- The color of each note made it easier to tell which note it was (distinguishing e.g. C from D from E).
- The color of each note helped me tell whether it was higher or lower in pitch.
- The changing colors felt overwhelming or distracting while playing.
- The color changes felt unnecessary – I could have played just as well without them.

Session bookkeeping (participant ID, starting condition, realized condition sequence, levels completed) and questionnaire responses were saved to two tabs of a shared Google Sheets document, `SessionData` and `Formularantworten 1`, described further in the following section.

5.2 Implementation of the Evaluation

Two separate logging pipelines feed two separate parts of this chapter.
Technical logging, feeding Section 5.3, works as described in Section 3.3.3. `EvaluationLogger` runs per active-playing note and accumulates peak values across the note's frames. On the completion, it writes a single per-tap summary row to CSV alongside the ground-truth note and instrument data. A separate time-series mode records every frame of one selected tap in full. The complete STFT spectrum, chroma vector and other per-frame values, rather than just the tap's peaks. It is used specifically for the within-note figures (spectrogram, spectral centroid decay) that a single aggregated row cannot represent. No live user interaction is needed to generate either. Both are recorded automatically every time a note plays during any session, including internal testing.
Session logging, feeding Section 5.4, is handled separately by `StudySessionManager`, which records session-level bookkeeping per participant, participant ID, starting condition, the "Old/New" sequence across the levels played and how many levels were completed. Then forwards it, together with the participant's questionnaire responses, to a Google Form endpoint via a small Apps Script web app, landing in a Google Sheet for analysis. This pipeline is intentionally separate from the technical CSV logging, because one captures what the analysis methods measured, the other captures what the participant experienced and reported and the two are joined only at analysis time via the shared condition data both recorded.
Logged data was checked for internal consistency before analysis. Participant ID gets compared on both logged data so that the rows are matching.

5.3 Technical Evaluation

This section asks two separate questions from two separate datasets. Section 5.3.1 asks "For a single instrument, how accurately does each method track pitch across different notes?". Sections 5.3.2–5.3.3 asks "For a single, fixed note, how much does each method's output change purely as a function of which instrument played it and does `CombinedResponder`'s normalization help?". Section 5.3.4 then reports the measured real-time cost of the pipeline itself.

5.3.1 Pitch-Tracking Accuracy (Single Instrument)

This first evaluation isolates a single instrument, a marimba and asks how accurately each method tracks pitch across four notes (C, G, A, B; $n = 5$–8 taps per note), using per-tap summary logging together with one full time-series capture of note G for the STFT mode which is necessary to produce the required spectral representations.Onset detection is excluded from this comparison, because spectral flux sums magnitude change across all bins into a single scalar and by construction carries no frequency information, so there is no pitch signal to evaluate.
Peak RMS/envelope cluster within 0.13–0.18 across all four notes with no pitch-ordered separation (Figure 5.4), confirming amplitude carries no pitch information, as the design in Section 3.4.1.2 assumes. Samples: $n = 5$ for C, $n = 8$ for G, $n = 5$ for A, $n = 5$ for B (23 taps

total), ordered by ascending true fundamental frequency. Standard deviation (SD) is ≈ 0.000 for C, G and B. Note A shows minor variance (SD $= 0.020$) from one tap at lower tap intensity.

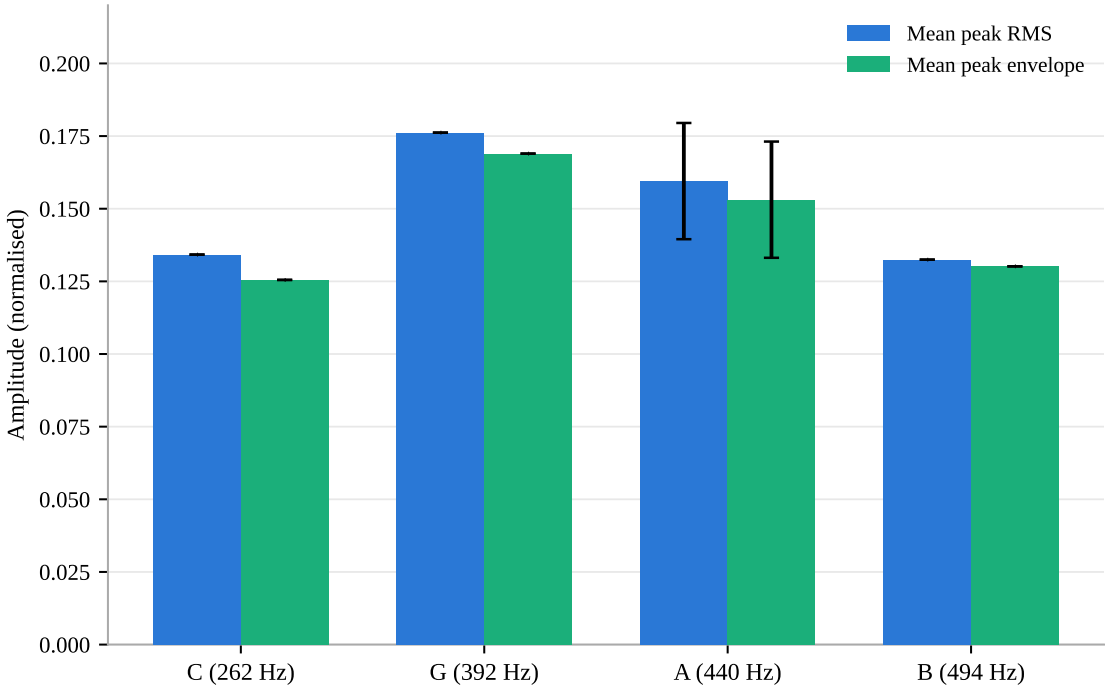


Figure 5.4: RMS/envelope peak value by note, marimba only. No pitch-ordered separation is visible.

FFT dominant-frequency peak-picking is unreliable even for a single instrument (Figure 5.5). Dominant frequency (argmax of the Hann-windowed FFT magnitude spectrum) was sampled at frame 3 post-onset (≈50 ms). Standard deviation is 0.0 Hz for all notes across all repetitions, confirming the results are deterministic properties of the audio samples rather than measurement noise. C (true 262 Hz) is misidentified at 469 Hz ($+78.9\%$ error) because its second harmonic outweighs the fundamental, which is the same bin-instability problem noted in Section 3.4.1.1. While G, A and B show a consistent -10% systematic underestimation from bin quantization error (bin width ≈21.5,Hz at 44,100,Hz / 2048 samples), a separate resolution limit of the FFT approach itself.

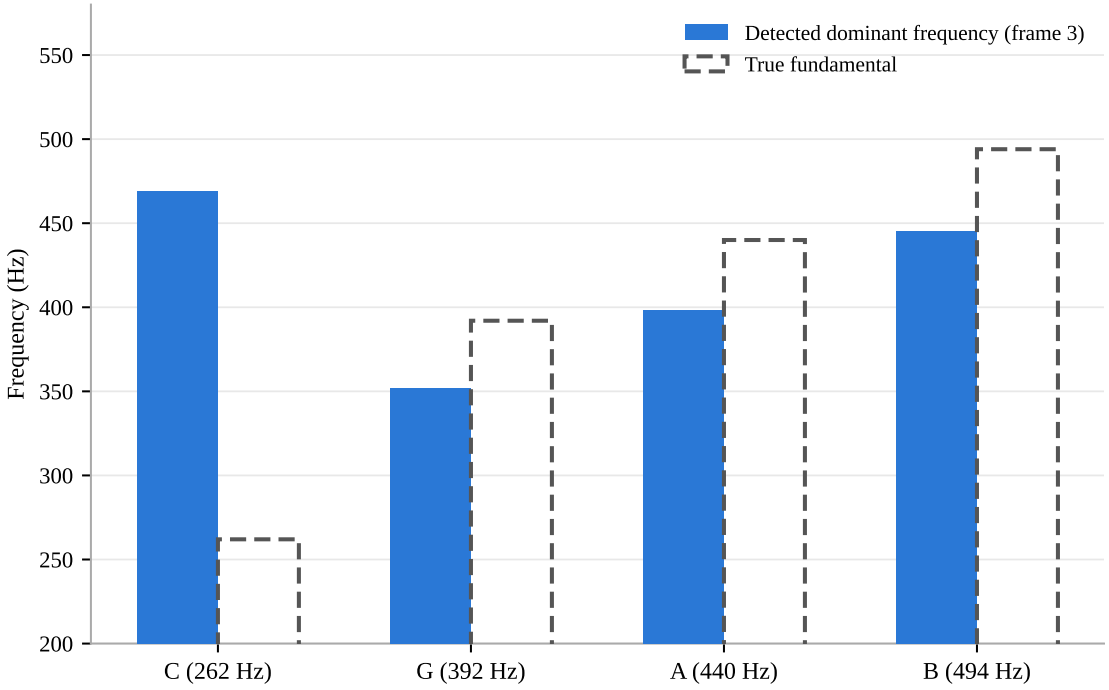


Figure 5.5: FFT dominant-frequency detection by note, marimba only. C is misidentified via second-harmonic dominance; G/A/B are underestimated from bin quantization.

Table 5.1: FFT-detected dominant frequency (argmax of Hann-windowed FFT magnitude spectrum, frame 3 post-onset) versus true fundamental, marimba only. Bin width ≈ 21.5 Hz at 44,100 Hz / 2048 samples.

Note	True fundamental	Detected mean (SD)	Error (Hz)	Error (%)	Probable cause
C	262 Hz	468.8 Hz (± 0.0)	$+206.8$	$+78.9\%$	2nd harmonic dominance from pitch-shift artifact
G	392 Hz	351.6 Hz (± 0.0)	-40.4	-10.3%	FFT bin quantization + pitch-shift spectral shift
A	440 Hz	398.4 Hz (± 0.0)	-41.6	-9.5%	FFT bin quantization + pitch-shift spectral shift
B	494 Hz	445.3 Hz (± 0.0)	-48.7	-9.9%	FFT bin quantization + pitch-shift spectral shift

Chroma performs better, though not perfectly (Figure 5.6). Mean normalized chroma feature vectors at frame 3 post-onset for notes C, G, A and B ($n = 5$–7 taps per note, SD ≈ 0) show that chroma correctly identifies C and A outright, while G is detected one semitone sharp (G#) and B produces a near-miss (true bin wins over the runner-up by only $\Delta = 0.005$). Both are attributable to the linear-interpolation pitch-shifting used to generate these samples, even after the $+1.88$ semitone correction described in Sections 3.4.1.4 and 4.3. This specific one-semitone failure on G is the reason the correction exists in the first place and is examined across more instruments in Section 5.3.2.

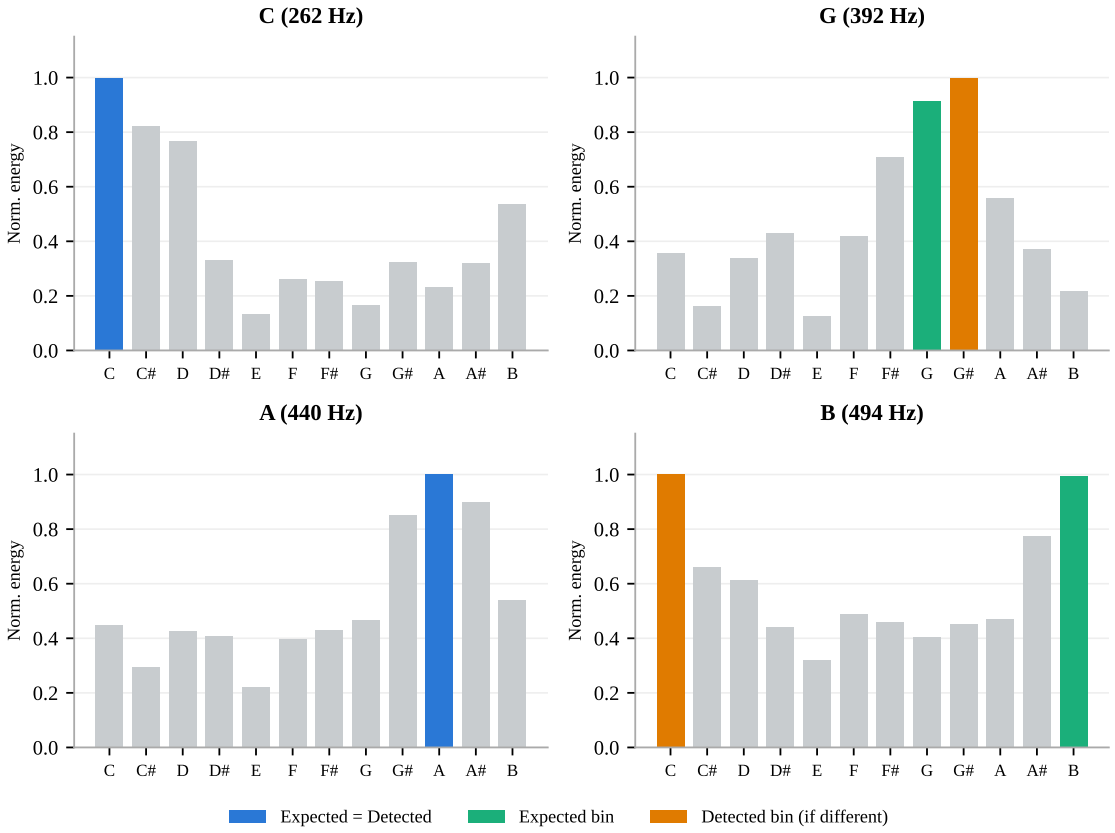


Figure 5.6: Mean normalised chroma feature vectors resulting from linear interpolation pitch-shifting of marimba samples.

Table 5.2: Chroma pitch class detection at frame 3 post-onset, marimba only (mean over $n = 5$–7 taps per note, SD ≈ 0.000).

Note	True pitch class	Detected peak	2nd highest	Margin	Verdict
C	C (bin 0)	C (bin 0) — 1.000	C# (bin 1) — 0.822	$+0.178$	Correct
G	G (bin 7)	G# (bin 8) — 1.000	G (bin 7) — 0.915	-0.085	Off by 1 semitone
A	A (bin 9)	A (bin 9) — 1.000	A# (bin 10) — 0.900	$+0.100$	Correct
B	B (bin 11)	C (bin 0) — 1.000	B (bin 11) — 0.995	-0.005	Near-miss ($\Delta = 0.005$)

A single-tap spectrogram of note G (Figure 5.7) shows why chroma still outperforms raw FFT peak-picking despite the semitone error above. The fundamental (≈345 Hz) and its harmonics (≈690 Hz, ≈1335 Hz) decay together and stay clearly separated in the magnitude spectrum, so folding harmonics into pitch classes is far more robust than picking whichever bin happens to be momentarily loudest.

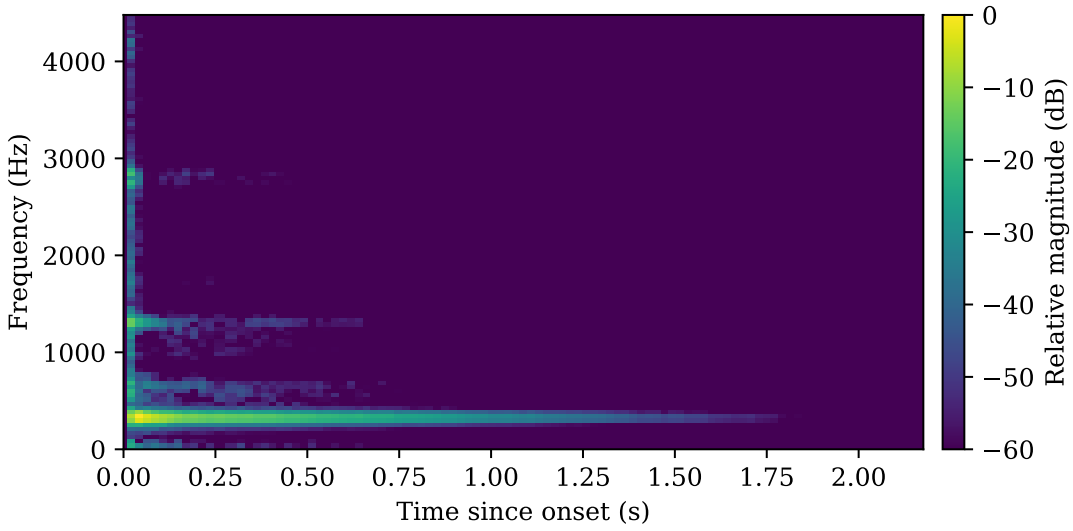


Figure 5.7: STFT magnitude spectrogram of a single tap of note G (392 Hz, marimba), computed from rolling 1024-sample Hann-windowed FFT frames (≈43 Hz bin width, 44,100 Hz sample rate). Colour shows relative magnitude in dB, normalized to the frame's own peak.

Spectral centroid (magnitude-weighted mean frequency), sampled at frames 1, 3, 10, 20 and 30 post-onset, shows no consistent pitch-ordered separation either (Figure 5.8). Values range 860–3390 Hz across all notes and note rankings cross repeatedly across the sampled frames. At frame 10, B drops to its minimum (1413 Hz) while A reaches its maximum (3390 Hz), even though B has the highest true fundamental (494 Hz). This confirms spectral centroid encodes instantaneous decay stage rather than pitch-class identity for these pitch-shifted marimba samples.

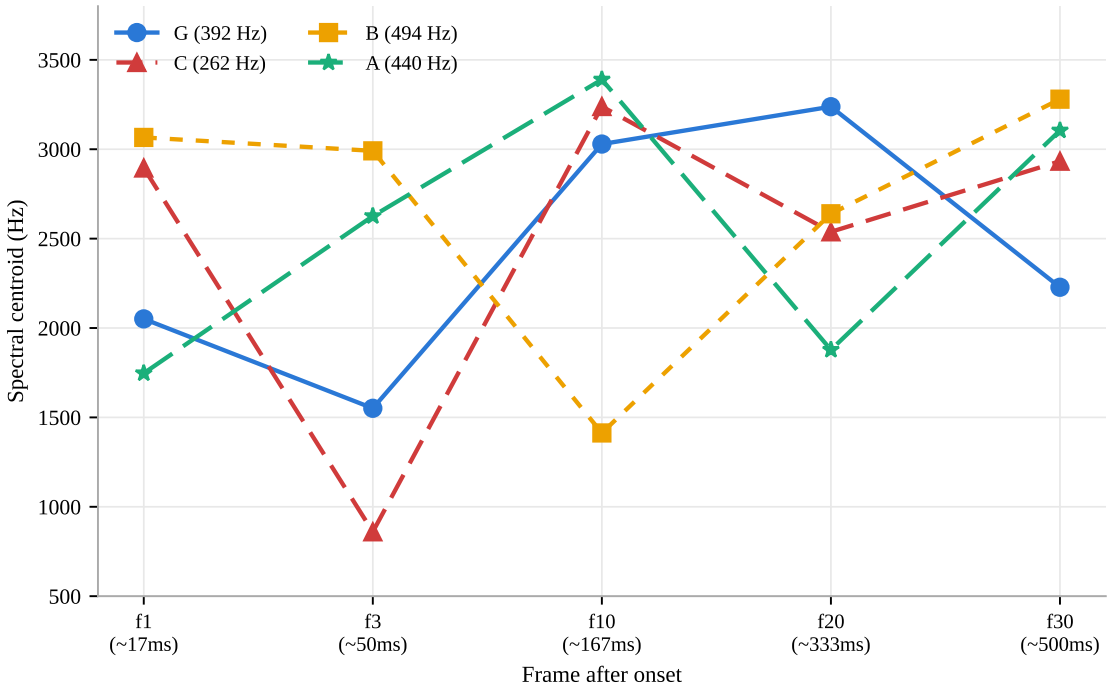


Figure 5.8: Spectral centroid (magnitude-weighted mean frequency) at five fixed post-onset frames, for four notes played on the same instrument.

5.3.2 Chroma Correction Generalization Across Instruments

`ChromaExtractor` applies a single global correction, `PITCH_CORRECTION_SEMITONES = 1.88`, derived empirically from one G-note observation on marimba. Testing the same fixed note (D) across four instruments (marimba, guitar, percussion, bass; $n \approx 13$–15 taps each) shows this correction is not adjustable to all instruments (Table 5.3).

Table 5.3: Chroma pitch class detection by instrument (fixed note D)

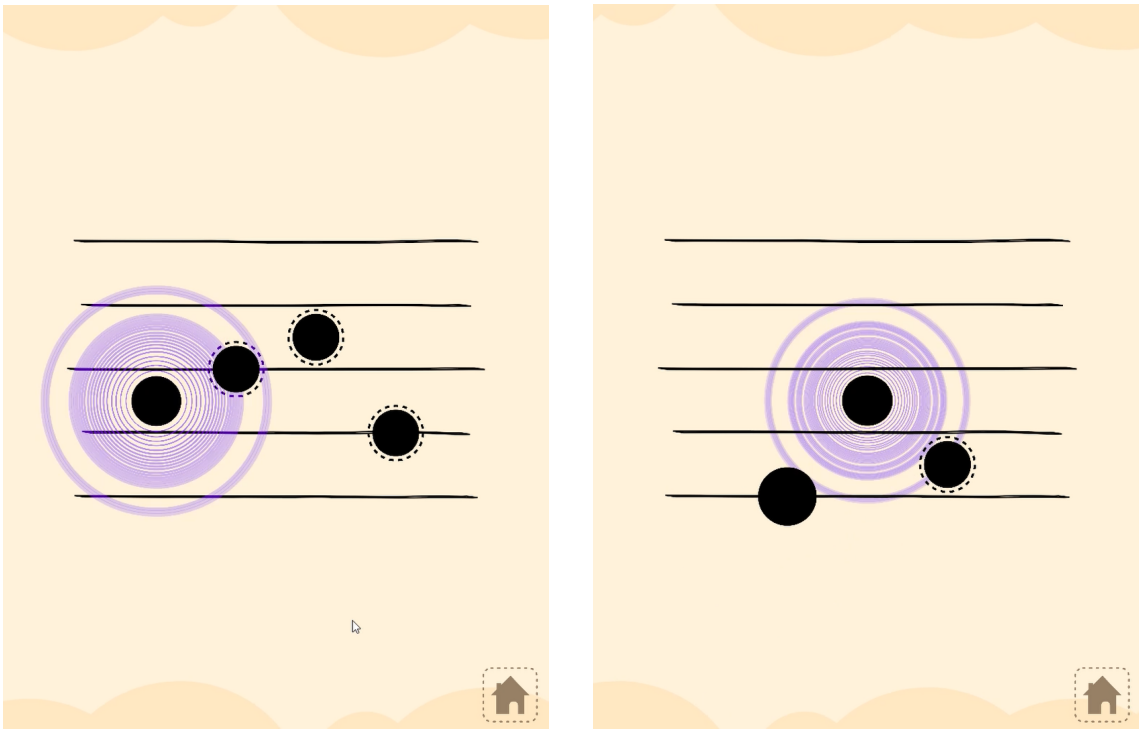
instrument	detected	semitones off	correct taps	conf. (true)	conf. (detected)	unstable
Marimba	D#	1	0/15	0.191713	1.000000	no
Guitar	D#	1	0/13	0.064162	1.000000	no
Percussion	D	0	13/15	0.965173	0.965173	yes
Bass	G#	6	0/15	0.018493	0.992667	yes

Marimba and guitar are both detected as D#, one semitone sharp, on every single tap and confidently so (detected-bin confidence 1.000 vs. the true D bin's 0.191713 and 0.064162 respectively), which is a systematic, not random, error. Percussion is mostly correct (13/15 taps) but occasionally unstable within a single instrument. Bass is detected as G#, six semitones off and is unstable across taps. This is consistent with the correction being tuned to compensate for one instrument's specific spectral profile (marimba's own pitch-shift artifact) rather than a property of the pitch-shifting method in general. It happens to also work for marimba's and guitar's *direction* of error but not its *magnitude* and does not hold at all for bass. This

finding is reported as-is rather than used to re-derive a per-instrument correction or patch `PITCH_CORRECTION_SEMITONES`, since the component exists to demonstrate a comparison point, not to become production-accurate.

5.3.3 Timbre Robustness Across Instruments and the Combined Responder's Fix

The same dataset as Section 5.3.2 (fixed note D, four instruments) additionally evaluates the other four methods, plus the `drive` value `CombinedResponder` uses in place of raw envelope (Section 4.3). Figure 5.9 shows what this timbre-dependence looks like in practice: the New system's visual response to the same note played on guitar versus marimba.



(a) Guitar (b) Marimba

Figure 5.9: New system visual response to the same note played on two different instruments.

Plotting the individual per-tap latencies (Figure 5.10) rather than only the summary statistics in Table 5.4 shows the underlying distribution is close to binary rather than continuously jittery. For Marimba, Bass and Guitar, all but exactly one tap land at 0 ms latency and the mean/SD reported in the table are driven entirely by that single outlier tap (21.3 ms for Marimba and Bass, 192 ms for Guitar). Percussion is the only instrument with zero jitter on every tap. Onset timing is not moderately noisy across the board, it is exact almost all the time, scattered by rare, instrument-dependent misses.

Table 5.4: Onset detection reliability and latency by instrument (fixed note D)

instrument	n	detect rate	lat. mean (s)	lat. SD (s)	lat. max (s)	str. mean	str. SD
Bass	15	1.000000	0.001420	0.005500	0.021300	0.354766	0.024733
Guitar	13	1.000000	0.014769	0.053251	0.192000	0.482772	0.020318
Marimba	15	1.000000	0.001420	0.005500	0.021300	0.532017	0.170984
Percussion	15	1.000000	0.000000	0.000000	0.000000	0.481509	0.027362

Table 5.5: RMS / Envelope – raw vs. normalized drive

instrument	peakEnvelope_mean	peakEnvelope_std	peakDrive_mean	peakDrive_std
Bass	0.277063	0.007866	0.298258	0.008468
Guitar	0.211475	0.000052	0.304391	0.000080
Marimba	0.288761	0.005420	0.377592	0.007089
Percussion	0.167604	0.000640	0.218743	0.000836

- **RMS/envelope vs. drive.** Cross-instrument coefficient of variation drops from 24.1% (raw peak envelope) to 21.7% (normalized drive), Table 5.5. The normalization helps, but does not equalize loudness across instruments, as stated in Section 4.3.
- **FFT dominant frequency.** Highly different per instrument for the same note (Table 5.6): 140.6 Hz (bass), 281.3 Hz (guitar), 609.4 Hz (marimba) and an implausible 20,578.1 Hz for percussion, where the largely inharmonic, noise-like spectrum has no clear fundamental for peak-picking to lock onto at all.
- **Chroma pitch class.** Covered in Section 5.3.2.
- **Onset detection.** Onset fires on every tap for all four instruments (100% detection rate, Table 5.4), the one method that holds up unconditionally across timbre, but latency and consistency vary. The percussion is frame-exact every time (0 ms latency, 0 jitter, as expected for a sharp transient), while guitar's softer attack produces both higher mean latency (≈15 ms) and much larger jitter (up to 192 ms on one tap).
- **STFT bands.** Mid-band energy varies moderately across instruments (Coefficient of variation (CV 50.3%, Table 5.7) while high-band energy varies far more (CV 178.9%). This is because it is driven mainly by percussion's much higher high-frequency content relative to the other three. The RMS-relative normalization does not remove instrument-specific spectral shape.

Read together, of the five single-method extractors, onset detection is the only one that is essentially timbre-robust as tested. The other four each show clear, instrument-dependent failure modes, ranging from confidently "wrong by a fixed offset" (chroma on marimba/guitar) to unusable (FFT on percussion). `CombinedResponder`'s per-clip normalization measurably reduces the loudness-driven version of this problem without eliminating it and sidesteps the pitch-driven version entirely by using ground-truth pitch class for color instead of any detected

Table 5.6: FFT dominant frequency by instrument (fixed note D)

instrument	mean	std	min	max
Bass	140.600000	0.000000	140.600000	140.600000
Guitar	281.300000	0.000000	281.300000	281.300000
Marimba	609.400000	0.000000	609.400000	609.400000
Percussion	20578.100000	0.000000	20578.100000	20578.100000

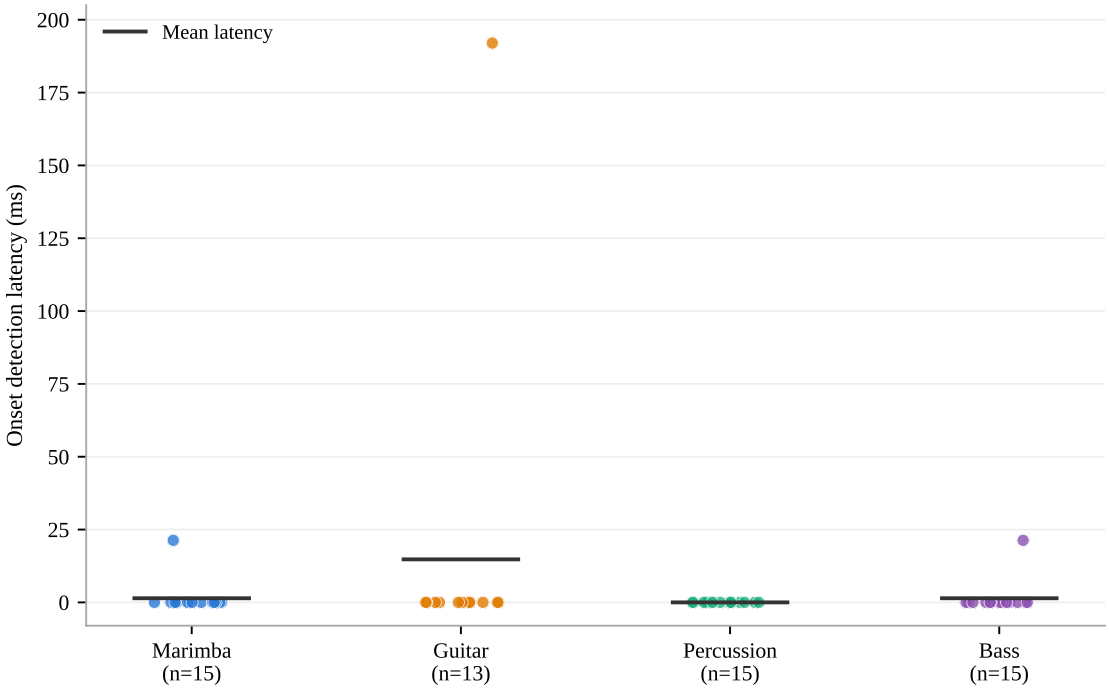


Figure 5.10: Per-tap onset detection latency by instrument (fixed note D), individual taps jittered horizontally for visibility. Solid black bar marks the mean per instrument.

value (Section 4.3). It does not and was never intended to, fix chroma, FFT, or STFT detection accuracy themselves.

5.3.4 Real-Time Performance

The double-buffer lock introduced in Section 4.4 was measured directly rather than assumed safe. Lock duration was captured with a `System.Diagnostics.Stopwatch` across 161 measurement windows of 200 `OnAudioFilterRead()` callbacks each (≈12.5 minutes of audio total). And those were compared against the fixed DSP buffer deadline of 23,220 µs (1024 samples at 44,100 Hz, matching the sample rate used throughout the rest of this thesis). Figure 5.11 and Table 5.8 summarize the result. The average lock duration was 0.77 µs, which is far below the buffer deadline. Periodic maximum spikes of up to 3,677 µs occurred in 31.7% of windows, but even the largest observed spike stayed at only 15.8% of the DSP buffer period. These spikes are attributed to OS scheduling jitter on Windows rather than a conflict from the main thread itself, since their magnitude and periodicity did not correlate with note density during capture.

Table 5.7: STFT mid/high band energy by instrument (fixed note D)

instrument	peakStftMid_mean	peakStftMid_std	peakStftHigh_mean	peakStftHigh_std
Bass	0.039838	0.016115	0.001619	0.002017
Guitar	0.160455	0.020893	0.003436	0.003809
Marimba	0.117340	0.000000	0.001701	0.000230
Percussion	0.086390	0.000000	0.078300	0.000000

No audio dropouts were observed during data collection.

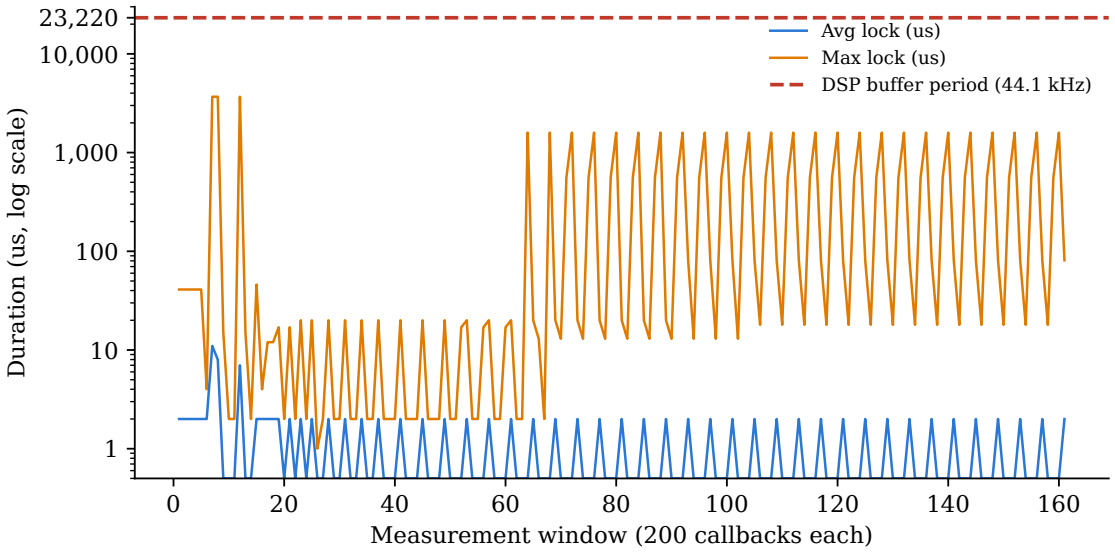


Figure 5.11: Double-buffer lock duration across 161 measurement windows of 200 `OnAudioFilterRead()` callbacks each. Average lock duration (blue) and maximum observed per window (orange), plotted on a logarithmic scale against the fixed DSP buffer deadline (red, dashed).

5.4 User Survey

This section presents the questionnaire results collected via the pipeline described in Section 5.2, comparing the existing static visual system ("Old") against the new particle-based system driven by `CombinedResponder` ("New"). Of 17 recorded sessions and 16 form responses, 15 participant IDs appear in both and were used for the analysis below. Two participants played past the survey gate without ever submitting a questionnaire and one form response had no matching session, so all three were excluded. Four tests were made in person, the other participants downloaded the app themselves, played it remotely and did the survey without in person observation. The Google Form itself asks participants to compare "version A" and "version B". The confirmed mapping is A = Old, B = New and all figures below already use Old/New directly.

Table 5.8: Double buffer lock duration across 161 measurement windows (200 `OnAudioFilterRead()` callbacks each), referenced against the DSP buffer period at 44.1 kHz.

Metric	Avg lock (µs)	Max lock (µs)
Mean across all windows	0.77	412.24
Median	0.0	20.0
Standard deviation	1.45	724.34
Minimum	0	1
Maximum observed	11	3,677
DSP buffer period (44.1 kHz, 1024 samples)		23,220 µs
Worst case as % of DSP buffer		15.84%
Windows with max $>$ 100 µs (spikes)		51 / 161 (31.7%)

5.4.1 Paired Preference Results

Participants answered seven paired Old/New questions covering pitch identification, duration perception, perceived synchronization, whether the visuals stopped cleanly with the sound, match with expectation, enjoyment and overall preference (Figure 5.12). Pitch identification is the least ambiguous result, because 9 of 15 participants leaned toward New and only 2 toward Old. Perceived synchronization leans the other way, 7 of 15 toward Old versus 1 toward New, indicating the New system's particle timing was perceived as slightly less synchronized than the existing static system, which is a legitimate limitation to weigh against the synchronization design in Section 4.4, not only a result to just disregard. Duration perception and expectation match were roughly balanced, with a majority selecting "No preference" on both. Enjoyment came out even (7 vs. 7) and overall preference tilted slightly toward Old (7 vs. 6), but both questions were sharply polarized rather than neutral. 13–14 of 15 respondents chose "Much" or "Slightly" rather than "No preference," so the more accurate reading is not that opinions were absent but that they were roughly evenly split and strongly held.

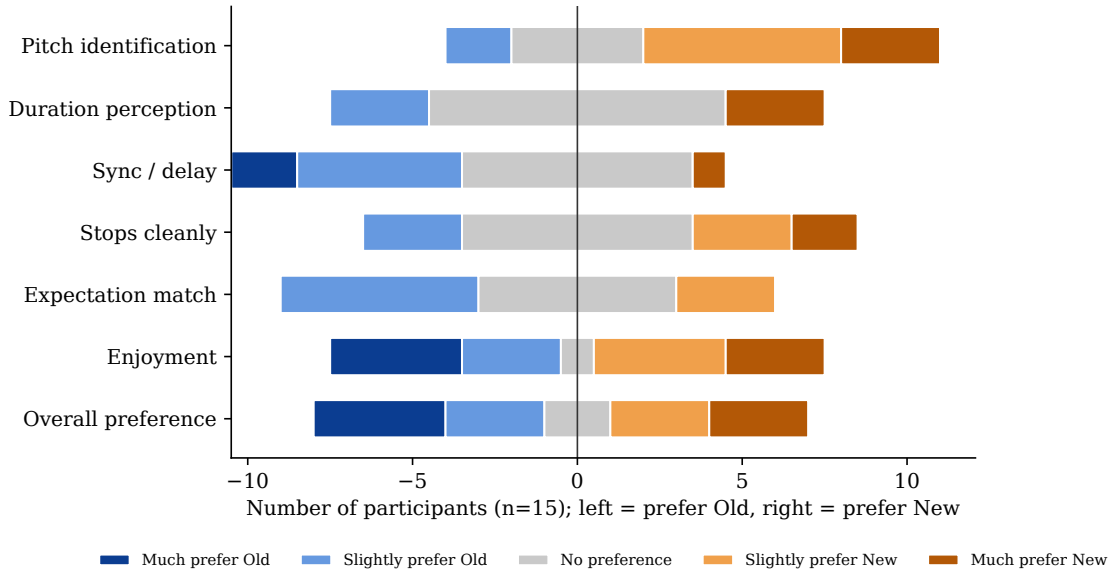


Figure 5.12: Diverging comparison of the seven paired Old/New survey questions ($n = 15$). Bars extend left for participants preferring Old and right for participants preferring New; grey segments centered on zero represent "no preference."

5.4.2 Color-Specific Ratings

Four additional 1–5 Likert statements asked about the note-color changes introduced with the New system (Figure 5.13). Note the scale direction, taken directly from the Google Form: $1 =$ agree, $5 =$ disagree. Participants leaned toward *agreeing* that color helped distinguish which note was being played (mean 2.87), but leaned toward *disagreeing* that color specifically helped judge pitch height (mean 3.67). This is a meaningful distinction rather than the same finding twice. The color appears to have helped identify *which* note it was, rather than it helped judge whether a note was higher or lower. Participants also leaned toward *disagreeing* that the color changes felt overwhelming or distracting (mean 3.80), which is a favorable result for perceptual load. At the same time, they leaned toward *agreeing* that the color changes were unnecessary (mean 2.53). So while color was not experienced as distracting, participants were not convinced it was needed either. Read together, this is a more mixed result than a clean "color helps" or "color hurts" conclusion. It appears useful for note identification without being distracting, but its perceived necessity is still in question, which is discussed in Section 5.5.

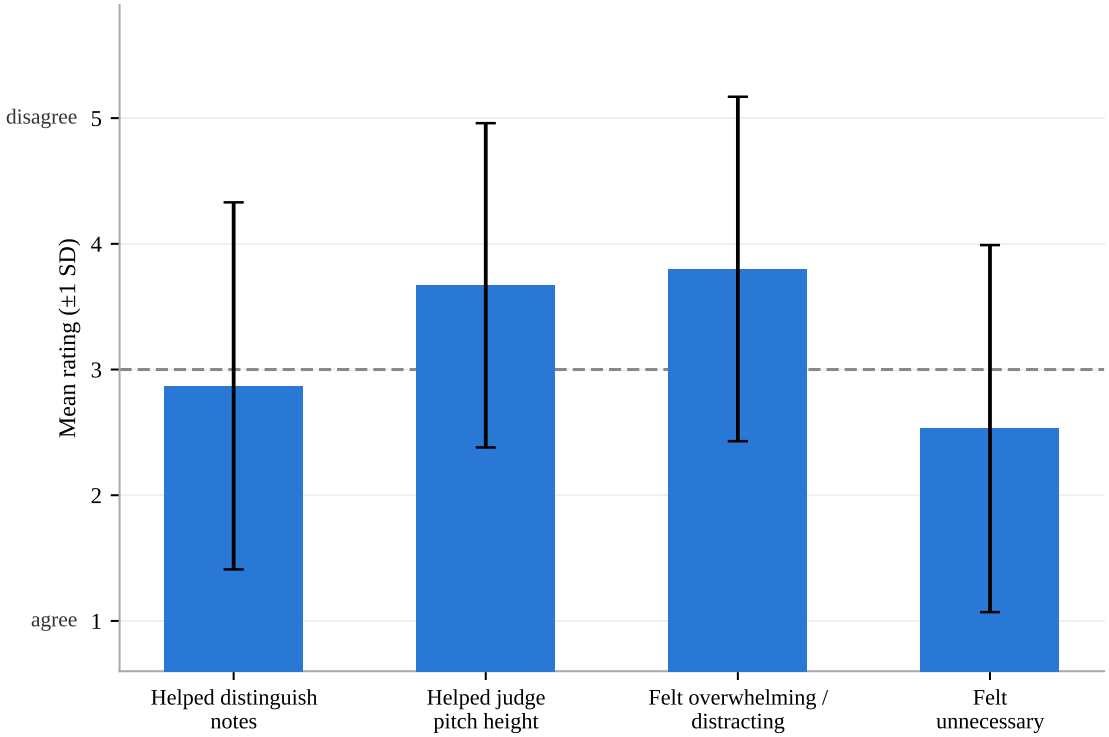


Figure 5.13: Mean rating (1–5 Likert, error bars = 1 SD) for four color-specific statements, $n = 15$. Dashed line marks the scale midpoint (3).

5.4.3 Breakdown by Prior Music Experience

As an exploratory check, responses were additionally grouped by participants' self-reported prior music experience ("None", "Some", "Extensive"; $n = 5$ per group, Figure 5.14). This is not a statistically tested effect and the subgroup sizes are small, so it is reported only as a pattern worth noting rather than a finding. Participants with no prior music experience leaned toward New on both overall preference and enjoyment ($+0.40$ each, on a -2 to $+2$ scale), while participants with "Some" or "Extensive" experience leaned toward Old on both (Some: $-0.40/-0.40$; Extensive: $-0.40/-0.20$). The two color-rating items use the same $1 =$ agree, $5 =$ disagree scale as Section 5.4.2, so higher means *less* agreement. On "felt overwhelming or distracting," the "Extensive" group leans most strongly toward disagreement (4.6), "None" is intermediate (3.8) and "Some" leans closest to the neutral point (3.0). The opposite ordering of what a simple "more experience, more distraction" assumption would predict. On "felt unnecessary", "Extensive" again leans furthest from agreement (3.2), while "None" and "Some" both lean toward agreeing it was unnecessary (2.4 and 2.0), with "Some" leaning most strongly of all three. Read together, participants with more music background still preferred the Old condition overall, but they were not the group most bothered by or most dismissive of the color cues specifically. If anything the "Extensive" group is the least critical of the color feature on both items, which does not fit neatly with their Old-leaning overall preference.

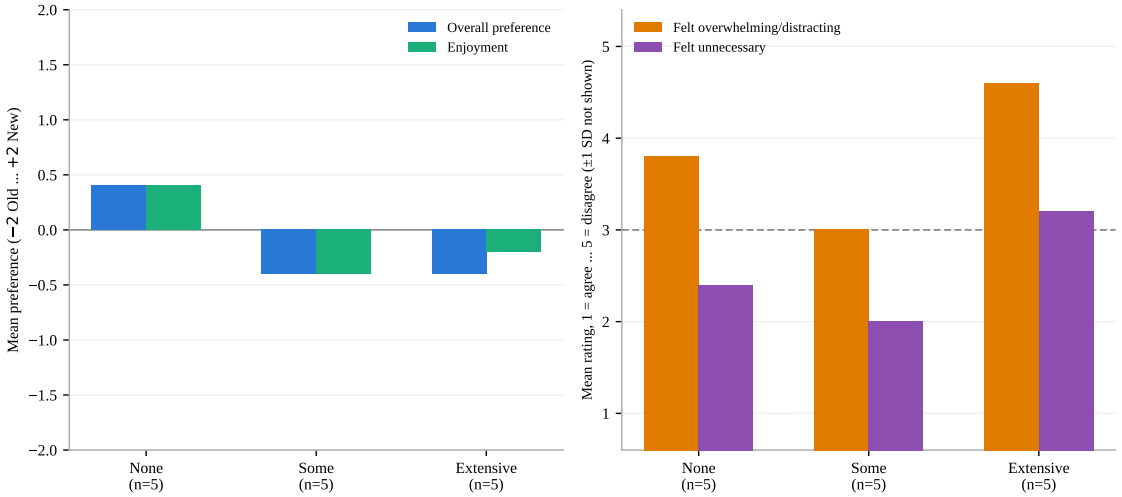


Figure 5.14: Mean Overall Preference and Enjoyment score (-2 = much Old $\ldots +2$ = much New, left panel) and mean rating for "felt overwhelming/distracting" and "felt unnecessary" (1–5 Likert, right panel), grouped by prior music experience (None/Some/Extensive, $n = 5$ each).

5.5 Evaluation and Discussion

Taken together, the five single-method extractors are not accurate enough on their own to drive pitch-based visuals reliably once timbre varies. Chroma comes closest, correctly identifying two of four notes for a single instrument (Section 5.3.1) and generalizing partially, but not consistently, once tested across instruments (Section 5.3.2); FFT and STFT-band energy are considerably less reliable once timbre changes (Section 5.3.3). Onset detection is the one exception, holding up unconditionally across all four instruments, but by construction it only signals that a note began, not which note. None of this is a performance problem: all five methods run comfortably inside the real-time budget (Section 5.3.4), so the limiting factor is signal accuracy, not computational cost. This is precisely why `CombinedResponder` does not rely on any single method's detected pitch for color, but uses the ground-truth pitch class already known from gameplay instead (Section 4.3).

Some of the measured pitch-tracking error is likely attributable to the marimba note samples themselves rather than to the extraction methods: the same samples were generated by linear-interpolation pitch-shifting from a small set of recordings and several of the more surprising results (C's second-harmonic misidentification, chroma's fixed one-semitone bias) plausibly reflect artifacts of that shifting process rather than a property of the underlying method. A methodological lesson for repeating this evaluation is to validate synthetic sample quality against real per-note recordings first, so that method error and sample-generation error are not conflated.

Because `CombinedResponder` sidesteps detected pitch for color entirely, participants in the user survey were never actually exposed to the accuracy problems measured in Section 5.3, which may explain why the survey results do not obviously track the technical findings. Pitch

identification leaned toward New (Section 5.4.1), consistent with color helping distinguish notes, but perceived synchronization leaned toward Old. This is unlikely to be explained by the double-buffer lock itself, which Section 5.3.4 measured as negligible (0.77 µs average). A more plausible technical explanation lies in a structural difference between the two systems. The existing Old visual (`ToneBlock.PlaySound()`) fires synchronously from the main-thread input callback at note-schedule time, with no dependency on the captured audio signal, so it cannot itself lag behind playback. `CombinedResponder` deliberately reads the actual audio signal instead of schedule time to avoid a different failure mode, which requires onset detection to first confirm a note has started; Section 5.3.3 measured this as exact on the large majority of taps but with rare latency spikes (up to 192 ms on one guitar tap). Participants may have noticed these occasional spikes and generalized them into an overall "less in sync" impression of New, even though New is the more audio-accurate of the two by design. Old's apparent advantage here may partly reflect that it is not derived from the audio at all, rather than being more precisely synchronized to it. It is equally possible this result instead reflects uncertainty about what "in sync" meant, or difficulty recalling the alternating conditions accurately by the time participants reached the questionnaire at the end of the session; the current data cannot separate these explanations from one another. The color-specific ratings were similarly mixed. Participants leaned toward finding color useful for note identification but not for judging pitch height. Also toward disagreeing that it was distracting while still leaning toward agreeing it was unnecessary (Section 5.4.2), which is a real, unresolved tension rather than a clear verdict either way.

Several participants' comments suggest they did not consciously register a difference between conditions at all. This is worth reporting rather than explaining away: it may mean the New system was unobtrusive enough not to interfere with the primary task, or that participants in this sample were not sufficiently invested in the comparison to form a strong opinion, or both. The current data cannot distinguish between these; participants were a convenience sample rather than learners with a personal stake in the outcome, which limits how far the survey results should be generalized to an actual learning context. A follow-up study with participants who are motivated learners rather than volunteers, told what is being compared and tested in person rather than largely remotely (Section 5.4), would be better positioned to isolate this effect; this is taken up further in the Future Work section.

Beyond sample size ($n = 15$), the evaluation has no no-visual baseline condition, so it can compare Old against New but cannot establish whether either visualization helps note learning at all relative to audio alone. Combined with the mixed color-necessity result above, this motivates the smaller color-scheme adjustments discussed in Future Work rather than a larger redesign.

6 Conclusion and Future Work

6.1 Conclusion

This thesis asked which audio-analysis methods are best suited for real-time visualization of pitch and note duration, considering accuracy, real-time performance and interpretability. No single method sufficed: chroma came closest but did not generalize across instruments (Section 5.3.2) and onset detection was the only timbre-robust method, at the cost of carrying no pitch information. Real-time performance was never the limiting factor, because all five methods ran comfortably inside budget (Section 5.3.4). So accuracy, not cost, is the real constraint identified here. This is why `CombinedResponder` uses ground-truth pitch class for color rather than any single detected value (Section 4.3), a design the evaluation supports. The user survey found mixed value in this approach. Color helped identify which note was played and was not experienced as distracting, though its necessity remained in question (Section 5.4). Real-time, audio-analysis-driven visualization of pitch and duration is doable today, provided the system does not depend on any single detection method in isolation.

6.2 Future Work

- **Timbre-robust pitch methods.** Testing methods designed for timbre robustness from the outset, such as the Harmonic Product Spectrum [15] or YIN [4], together with instrument-aware calibration, is a more direct fix than patching a single global correction constant (Section 5.3.2). Müller's [13] log-compression refinement for weak high-frequency transients is a concrete, unimplemented next step in the same direction.
- **Performance refinements.** Sharing one computed spectrum across FFT, chroma, onset and STFT (Section 4.5) and investigating the onset-latency spikes behind the perceived-synchronization result (Section 5.5), are both worth revisiting, though neither is a measured bottleneck.
- **A larger, targeted user study.** Specifically recruited beginners and children and an explicit no-visual baseline would address the limitations of the current $n = 15$ convenience sample (Section 5.5).
- **Refining the visual design.** Fading color and particle complexity as a player advances, narrowing the color range and separating note identity (hue) from pitch height onto different channels all follow directly from the mixed color-rating results in Section 5.4.2.

7 Use of AI Tools

This appendix documents the use of AI-based tools during the preparation of this thesis. The tool used throughout was Claude (Anthropic), primarily the `claude-sonnet-5` model, accessed via chat and via an agentic coding/file-editing interface (Claude Code / Cowork). Table 7.1 lists the main use cases together with a representative example prompt for each. All AI-assisted output, like text, figures, code and analysis, was reviewed by me, checked against primary sources (source code, raw evaluation data, or cited literature) where applicable and edited before inclusion in this thesis.

Tool/Model	Use case	Example prompt
Claude (claude-sonnet-5)	Language, grammar and clarity review of self-written text	"Are there some grammar issues?"
Claude (claude-sonnet-5)	LaTeX formatting and debugging (table overflow, `glossaries-extra` package/preamble errors, figure/table generation)	"This table is too long. it flows over the right side as in the picture seen"
Claude (claude-sonnet-5)	Building data figures and tables for the evaluation chapter from raw CSV logs and chart source data (e.g. RMS comparison, chroma vectors, onset latency)	"Build a table with this csv data and calculate the means and standard deviations?"
Claude (claude-sonnet-5)	Building the code framework for the analysis methods and visualization from equations and error logs	"Can you help me implement this algorithm/equation in C# and Unity Engine?"
Claude (claude-sonnet-5)	Statistical computation and interpretation support for the evaluation and survey results (e.g. CV, means, scale-direction check)	"What means CV?" / "How was that calculated?"
Claude (claude-sonnet-5)	Drafting assistance for specific passages, based on my own outline, data and prior draft text (e.g. Conclusion, Future Work, Abstract, individual result sentences)	"What would be your conclusion and future work in comparison to my draft text: "..."?"
Claude (claude-sonnet-5)	Organizing and documenting the raw evaluation CSV data for reproducibility	"Can you get all the csv files which were used for the evaluation and copy them in a new folder... I want to give them to the examiner so they can rebuild and check our data"

Table 7.1: AI-based tools used in the course of this thesis.

References

Literature Sources

[1] Juan Pablo Bello et al.: A Tutorial on Onset Detection in Music Signals. In: IEEE Transactions on Speech and Audio Processing 13.5, pp. 1035–1047. (Sept. 2005).

[2] Angel David Blanco, Simone Tassani, and Rafael Ramirez: Effects of Visual and Auditory Feedback in Violin and Singing Voice Pitch Matching Tasks. In: Frontiers in Psychology 12, p. 684693. Frontiers Media (2021).

[3] Judith C. Brown: Calculation of a Constant Q Spectral Transform. In: The Journal of the Acoustical Society of America 89.1, pp. 425–434. (1991).

[4] Alain de Cheveigné and Hideki Kawahara: YIN, a Fundamental Frequency Estimator for Speech and Music. In: The Journal of the Acoustical Society of America 111.4, pp. 1917–1930. (Apr. 2002).

[5] Mihaly Csikszentmihalyi: Flow: The Psychology of Optimal Experience. New York, HarperCollins (1990).

[6] Simon Duggal: Record, Mix and Master: A Beginner's Guide to Audio Production. Ed. by Paul J. Rogers. Cham, Springer (2024).

[7] Lutz Felbick: Rezension: Marcus Aydintan / Laura Krämer / Tanja Spatz (Hg.), Solmisation, Improvisation, Generalbass. Historische Lehrmethoden für das heutige Musiklernen. In: Zeitschrift der Gesellschaft für Musiktheorie (ZGMTH) 19.1, pp. 109–116. (2022).

[8] Gerald Goertzel: An Algorithm for the Evaluation of Finite Trigonometric Series. In: The American Mathematical Monthly 65.1, pp. 34–35. JSTOR (1958).

[9] Y. Joson: "Spectral Centroid Influence on Timbre and Musical Tension". North Carolina School of Science and Mathematics, unpublished manuscript. Dec. 2025.

[10] Hugo B. Lima, Carlos G. R. dos Santos, and Bianchi S. Meiguins: A Survey of Music Visualization Techniques. In: ACM Computing Surveys 54.7, 143:1–143:29. ACM (2021).

[11] Richard E. Mayer: "Multimedia Learning". In: *The Psychology of Learning and Motivation*. Ed. by Brian H. Ross. Vol. 41. San Diego, Academic Press (2002), pp. 85–139.

[12] Stephen McAdams: "The Perceptual Representation of Timbre". In: *Timbre: Acoustics, Perception, and Cognition*. Ed. by Kai Siedenburg et al. Vol. 69. Springer Handbook of Auditory Research. Cham, Springer (2019), pp. 23–57.

[13] Meinard Müller: Fundamentals of Music Processing: Using Python and Jupyter Notebooks. 2nd ed. Cham, Springer (2021).

[14] Andy Nealen, Adam Saltsman, and Eddy Boxerman: "Towards Minimalist Game Design". In: *Proceedings of the 6th International Conference on Foundations of Digital Games*. (2011), pp. 38–45.

[15] Alan V. Oppenheim and Ronald W. Schafer: Discrete-Time Signal Processing. In: Pearson Education Limited (2014).

[16] Lawrence R. Rabiner and Ronald W. Schafer: Introduction to Digital Speech Processing. In: Foundations and Trends in Signal Processing 1.1–2, pp. 1–194. Now Publishers (2007).

[17] Willi Schorrig et al.: "Tone – An Interactive Music Learning Experience". In: *2025 IEEE Conference on Games (CoG)*. IEEE. Lisbon, Portugal, (2025), pp. 1–2.

[18] John Sweller: Cognitive Load During Problem Solving: Effects on Learning. In: Cognitive Science 12.2, pp. 257–285. (1988).

Internet Sources

[19] International Telecommunication Union: Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level. Tech. rep. BS.1770-5. Accessed: 2026-07-28. ITU-R, 2023.

[20] Unity Technologies: AudioSettings.dspTime. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/AudioSettings-dspTime.html. (2022). Accessed: 2026-07-26.

[21] Unity Technologies: MonoBehaviour.OnAudioFilterRead. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/MonoBehaviour.OnAudioFilterRead.html. (2022). Accessed: 2026-07-26.

Acknowledgements

I would like to thank everyone who supported me during the preparation of this bachelor's thesis.
My sincere thanks go to my first examiner, Prof. Dr. Stefan Radicke, for the academic supervision of this thesis, as well as for the helpful suggestions and constructive criticism throughout the entire writing process. I would likewise like to thank my second examiner, Willi Schorrig, for providing his PhD project Tone as a framework to implement my thesis. Also for reviewing this thesis and for the valuable suggestions for improvement.
Special thanks go to Kai Elsässer and Pirmin Schweizer for their detailed and insightful feedback on this thesis, particularly on its structure, which meaningfully improved the final result. I would also like to thank my fellow students Lela Grösser, Bastian Hartmann and Dominik Müller for proofreading this thesis and for their feedback on language and phrasing.
Finally, I would like to thank my family and friends for their support and encouragement throughout my studies.